# HAWK: Fully Homomorphic Encryption Accelerator with Fixed-Word Key Decomposition Switching


Liang Kong
Ant Group
Beijing, China
kongliang.kong@antgroup.com

Shengyu Fan
State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS
Beijing, China
fanshengyu@iie.ac.cn

Xianglong Deng
State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS
Beijing, China
dengxianglong@iie.ac.cn

Lei Chen
Ant Group
Beijing, China
chenlei2014@mails.ucas.ac.cn

Guang Fan
Ant Group
Beijing, China
fanguang.fg@antgroup.com

Guiming Shi
Tsinghua University
Beijing, China
guimingthu@163.com

Yilan Zhu
Ant Group
Beijing, China
yilanzhu02@163.com

Geng Yang
Ant Group
Beijing, China
yanggeng.yang@antgroup.com

Shoumeng Yan*
Ant Group
Beijing, China
shoumeng.ysm@antgroup.com

Mingzhe Zhang*
Ant Group
Beijing, China
smartzmz@gmail.com



## Abstract

Fully Homomorphic Encryption (FHE) allows for direct computation on encrypted data, preserving privacy while enabling outsourced processing. Despite its compelling advantages, FHE schemes come with a significant performance penalty. Although recent cryptographic advances have introduced promising optimization technologies to mitigate computational burden, FHE accelerator designs face significant challenges in deploying these algorithmic optimizations. These challenges stem primarily from divergent computational precision requirements and escalating on-chip memory demands, leading to mismatches between algorithm and hardware constraints.

We propose HAWK, an FHE acceleration solution based on algorithm–hardware co-design. We begin with an in-depth analysis of the original key decomposition switching approach (KDS), identifying the main limitations of applying it to mainstream FHE accelerators. To address these issues, we propose the fixed-word key decomposition switching method (FW-KDS), significantly reducing computational overhead and memory requirements by up to 58%. We demonstrate how to effectively integrate this method into fixed-word accelerator designs and introduce several further optimizations. To address the expensive rounding operation inherent in the KDS method, we develop a new computational approach that reduces hardware costs while completely eliminating rounding errors. Furthermore, we adapt the underlying architecture to align with the computational demands of our algorithmic optimizations. To the best of our knowledge, this is the first exploration of deploying the KDS method in the fixed-word hardware accelerator. Experimental results demonstrate that HAWK achieves up to a 1.45× performance improvement with only a 4% area increase.



## CCS Concepts

• **Computer systems organization** → **Parallel architectures**; • **Security and privacy** → **Hardware security implementation**.

## Keywords

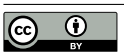
Fully homomorphic encryption, key-switching, accelerator

**ACM Reference Format:**
Liang Kong, Shengyu Fan, Xianglong Deng, Lei Chen, Guang Fan, Guiming Shi, Yilan Zhu, Geng Yang, Shoumeng Yan, and Mingzhe Zhang. 2025. HAWK: Fully Homomorphic Encryption Accelerator with Fixed-Word Key Decomposition Switching. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25), October 18–22, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3725843.3756123


## 1 Introduction

Fully Homomorphic Encryption (FHE) eliminates the risk of data exposure by keeping information encrypted even beyond user devices. This cryptographic technique enables remote servers to perform direct computations on encrypted data, allowing users to outsource

*Mingzhe Zhang and Shoumeng Yan are corresponding authors.





*MICRO '25, Seoul, Republic of Korea*



ACM ISBN 979-8-4007-1573-0/25/10
https://doi.org/10.1145/3725843.3756123

complex computations without sacrificing privacy. Among the various FHE schemes, the CKKS scheme [10, 14] emerges as a promising approach. CKKS facilitates the encryption of real and complex vectors, making it especially well-suited for applications in machine learning and data analytics [28, 30, 38, 40, 43, 53].

Despite its compelling advantages, current FHE schemes come with a substantial performance cost. Operations on encrypted data are typically several orders of magnitude slower than their plaintext equivalents. Various hardware solutions have been proposed to mitigate the computational overhead. These range from utilizing general-purpose CPU/GPU systems [3, 4, 11, 21, 26, 32, 48, 51] to designing specialized accelerators on FPGA platforms [2, 44, 49, 50, 52, 54] and developing custom ASIC solutions [22, 34, 35, 37, 45, 46].

Key-switching is a primitive operation in HE that is frequently invoked and highly computationally intensive. Previous FHE accelerators typically employ the widely used hybrid key-switching (HKS) method [29]. A new key-switching approach, key decomposition switching (KDS) [36], has been introduced to reduce computational complexity by utilizing the dynamic word length. It theoretically boasts superior asymptotic complexity. However, since the introduction of the KDS method, there has been a lack of comprehensive analysis and exploration regarding its practical efficiency and potential bottlenecks. Moreover, the compatibility of existing accelerator designs with this optimization method remains unexplored.

In this paper, we conduct a detailed analysis of the KDS method. Our investigation highlights four significant challenges encountered in practical implementation. **(i)** At small word lengths, the computational cost of the KDS method escalates rapidly, significantly underperforming compared to the HKS method. **(ii)** The dramatically increasing size of the evaluation key results in substantial memory demands. **(iii)** This approach is incompatible with previous fixed-word-length FHE accelerators. Accommodating the dual operational word lengths required by this method will impose significant hardware overhead for accelerators. **(iv)** The exact base conversion process integral to this method involves complicated rounding operations that existing accelerators do not support. Implementing this functionality with current computational approaches will incur substantial hardware costs.

Motivated by these considerations, we propose the method of fixed-word key decomposition switching (FW-KDS). This approach not only unifies the disparate word lengths required by the original KDS method, to accommodate fixed-word-length accelerator designs but also significantly reduces both computational and storage overheads. Additionally, we further demonstrate an effective application of the FW-KDS method in practical FHE acceleration designs. This includes introducing several optimization techniques to overcome limitations in method applicability. Experimental results show that HAWK delivers up to a 1.45× performance improvement with only a 4% area increase.

In summary, this paper makes the following major contributions:

- We present the first comprehensive analysis of the KDS method, shedding light on the various challenges it faces in practical accelerations. Although the method boasts superior theoretical algorithmic complexity, it encounters notable growth in both computational and storage demands when applied in practical scenarios.

**Table 1: CKKS parameter notation and descriptions.**

| Param. | Description |
|---|---|
| $N$ | Power-of-two polynomial ring degree. |
| $n$ | Maximum number of slots in the vector message, $n \leq \frac{N}{2}$. |
| $L$ | Maximum multiplicative level. |
| $L_{\text{eff}}$ | Maximum achievable level after bootstrapping. |
| $\ell$ | Current remaining multiplicative level. |
| $Q$ | Initial modulus, decomposed as $Q = \prod_{i=0}^{L-1} q_i$ (RNS). |
| $P$ | Auxiliary modulus, decomposed as $P = \prod_{i=0}^{\alpha-1} p_i$. |
| $\alpha$ | Limbs in the RNS decomposition of $Q$. |
| $\alpha'$ | Limbs in the RNS decomposition of $QP$ in KDS. |
| $\mathtt{dnum}$ | Number of limb groups after decomposition in HKS. |
| $\mathtt{dnum}'$ | Total number of limb groups used in KDS. |
| $z$ | Rounded integer in exact base conversion, formulated in Eq. 13. |
| $H$ | Converted Modulus, decomposed as $H = \prod_{i=0}^{T-1} h_i$. |
| $\tilde{Q}_i$ | Decomposed modulus of $Q$, satisfying $Q = \prod_{i=0}^{\mathtt{dnum}-1} \tilde{Q}_i$. |
| $Q_j$ | Decomposed modulus of $QP$, satisfying $QP = \prod_{j=0}^{\mathtt{dnum}'-1} Q_j$. |
| $A_{\text{InP},Q_j}$ | $Q_j$-limb of the inner product, formulated in Eq. 9. |
| $B_d$ | Bound for each $Q_j$-limb of inner product, formulated in Eq. 10. |
| $H_{ls}$ | Last converted modulus in FW-KDS, satisfying $H_{ls} = \prod_{i=0}^{t-1} q_i$ |

- We propose the fixed-word key decomposition switching (FW-KDS) that substantially reduces both the computational and storage costs associated with the original approach.
- We demonstrate the effective application of this method to FHE acceleration design, and introduce several optimizations that significantly decrease computational overhead while only slightly increasing memory demands.
- A new hardware-friendly computational approach for rounding operation is presented, along with a detailed explanation of the necessary architectural modifications to support the FW-KDS method in hardware implementations.

## 2 Background

In this section, we present a concise overview of the CKKS scheme. Table 1 provides the key symbols and notations used in this paper.

### 2.1 CKKS Basics

CKKS is founded on the computational hardness of the ring learning with errors (RLWE) problem [39] and enables fixed-point arithmetic over encrypted data. The *message* data type in CKKS is a real or complex vector of length $n$. Upon encryption, the message vector $\mathbf{m}$ is first encoded as a polynomial $\mathsf{P}_{\mathbf{m}}$ within the cyclotomic polynomial ring $\mathcal{R}_Q = \mathbb{Z}_Q[X]/(X^N + 1)$. Each entry in this ring is characterized as an integer polynomial of degree $N-1$ ($n \leq N/2$) with coefficients drawn from $\mathbb{Z}_Q$. The corresponding *ciphertext* $\mathbf{ct}$ comprises a pair of polynomials in $\mathcal{R}_Q$. The encryption process follows Eq. 1, employing a uniformly random polynomial $a$, a *secret key* $s$, and a small *error* polynomial $e$.

$$\mathbf{ct} = (b, a) = (-a \cdot s + \mathrm{P}_{\mathbf{m}} + e, a) \in \mathcal{R}_Q^2 \tag{1}$$

Complex homomorphic evaluations over ciphertexts are achieved through a sequence of primitive HE operations. Specifically, given two ciphertexts $\mathbf{ct}_0$ and $\mathbf{ct}_1$ where $\mathbf{ct}_i = (b_i, a_i) \in \mathcal{R}_Q^2$, the primitive HE operations can be summarized as follows:

**Plaintext Addition** (PAdd) incorporates a plaintext polynomial $\mathsf{P}_{\mathbf{m}'}$ from ring $\mathcal{R}_Q$ into an existing ciphertext. PAdd is computed as

$$\mathtt{PAdd}(\mathbf{ct}_0, \mathsf{P}_{\mathbf{m}'}) = (b_0 + \mathsf{P}_{\mathbf{m}'}, a_0) \tag{2}$$

**Homomorphic Addition** (HAdd) performs element-wise addition on two ciphertexts. This operation is conducted as

$$\mathtt{HAdd}(\mathbf{ct}_0, \mathbf{ct}_1) = (b_0 + b_1, a_0 + a_1) \tag{3}$$

**Plaintext Multiplication** (PMult) multiplies the ciphertext $\mathbf{ct}_0$ with a plaintext polynomial $\mathsf{P}_{\mathbf{m}'} \in \mathcal{R}_Q$ as

$$\mathtt{PMult}(\mathbf{ct}_0, \mathsf{P}_{\mathbf{m}'}) = (b_0 \cdot \mathsf{P}_{\mathbf{m}'}, a_0 \cdot \mathsf{P}_{\mathbf{m}'}) \tag{4}$$

Particularly, this operation allows for efficient scalar multiplication within the encrypted domain. A scalar can be regarded as a polynomial comprising solely a constant term.

**Homomorphic Multiplication** (HMult) performs ciphertext multiplication in the form of tensor product, followed by a *key-switching* (KeySwitch) procedure for relinearization. This process can be expressed as:

$$\begin{aligned}\mathtt{HMult}(\mathbf{ct}_0, \mathbf{ct}_1) =& (b_0 \cdot b_1, a_0 \cdot b_1 + a_1 \cdot b_0) \\ &+ \mathtt{KeySwitch}(a_0 \cdot a_1, \mathbf{evk}_{\text{mult}})\end{aligned} \tag{5}$$

where $\mathbf{evk}_{\text{mult}}$ denotes the *evaluation key* for multiplication. This operation preserves the encrypted structure while multiplying the underlying plaintext values.

**Homomorphic Rotation** (HRot) circularly shifts a message vector by $r$ slots and subsequently applies key-switching. Formally, the HRot result is given by

$$\mathtt{HRot}(\mathbf{ct}_0, r) = (\phi_r(b_0), \mathbf{0}) + \mathtt{KeySwitch}(\phi_r(a_0), \mathbf{evk}^{(r)}_{\text{rot}}) \tag{6}$$

where $\phi_r$ represents an automorphism applied to a polynomial, and $\mathbf{evk}^{(r)}_{\text{rot}}$ denotes the evaluation key specifically utilized for the rotation by amount $r$. The *automorphism* $\phi_r$ induces a permutation of polynomial coefficients, mapping the $i$-th coefficient to the position indexed by $i \cdot 5^r \bmod N$.

## 2.2 Ring Polynomial Functions

**Residue number system (RNS):** The polynomial modulus $Q$ in homomorphic encryption (HE) is typically a very large integer, often consisting of thousands of bits. To reduce computational complexity, the Residue Number System (RNS) [5, 13] is commonly used. We decompose the large modulus $Q$ into a product of $L$ smaller, word-sized prime moduli, expressed as $Q = \prod_{i=0}^{L-1} q_i$. The set $\mathcal{B} = \{q_0, q_1, \ldots, q_{L-1}\}$ forms an *RNS basis* for $Q$. Any polynomial $[a]_Q$ can then be uniquely represented as a tuple $([a]_{q_0}, [a]_{q_1}, \ldots, [a]_{q_{L-1}})$, with each element called a polynomial *limb*. This decomposition allows polynomial operations in $\mathcal{R}_Q$ to be carried out as $L$ independent operations on the individual limbs.

**Number-theoretic transform (NTT):** Naive polynomial multiplication in $\mathcal{R}_Q$ exhibits a complexity of $O(N^2)$. NTT, a modular-arithmetic variant of the Fast Fourier Transform, can efficiently reduce the overall complexity to $O(N \log N)$. In the NTT domain, also referred to as the *evaluation representation*, polynomial multiplication is transformed into an element-wise multiplication. Consequently, polynomials are typically maintained in their evaluation form for optimal performance, unless otherwise specified. When necessary, the inverse NTT (INTT) is employed to revert the polynomial to its original *coefficient representation*.

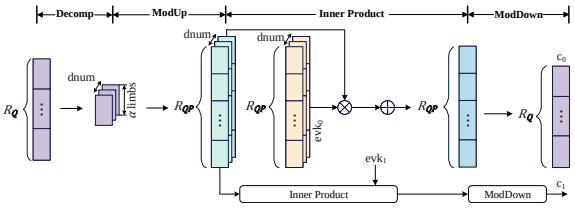



**Figure 1: Computational workflow of the HKS method. The generation of $c_1$ follows the same procedure as that of $c_0$.**

**Base conversion (BConv):** This function facilitates the matching of RNS basis between two polynomials characterized by distinct prime moduli. Consider a polynomial $\langle \tilde{a} \rangle_{\mathcal{B}}$ represented in an RNS basis $\mathcal{B} = \{q_0, q_1, \cdots, q_{L-1}\}$. The conversion to an alternative RNS representation $[\tilde{a}]_{\mathcal{A}}$, can be formally expressed as:

$$\mathtt{BConv}\left([\tilde{a}]_{\mathcal{B}}, \mathcal{B} \rightarrow \mathcal{A}\right) = \left\{ \left[ \sum_{i=0}^{L-1} \left[ [\tilde{a}]_{q_i} \cdot \hat{q}_i^{-1} \right]_{q_i} \cdot \hat{q}_i \right]_{p_j} \right\}_{j=0}^{k-1} \tag{7}$$

where RNS basis $\mathcal{A} = \{p_0, p_1, \cdots, p_{k-1}\}$, and constant $\hat{q}_i = \prod_{j \neq i} q_j$. Additionally, BConv requires the input polynomial $\langle \tilde{a} \rangle_{\mathcal{B}}$ to be expressed in coefficient representation.

## 2.3 Bootstrapping

To support complex programs with significant multiplicative depth, bootstrapping is introduced to refresh the level budget of ciphertexts [12]. The bootstrapping procedure consists of four steps: ModRaise, H-IDFT, EvalMod, and H-DFT. In ModRaise, the multiplicative level of ciphertext is raised to its maximum value. The H-IDFT step applies a homomorphic inverse discrete Fourier transform to the ciphertext, preparing it for the subsequent EvalMod process. EvalMod performs a homomorphic modulus reduction to reduce the noise in the ciphertext. The H-DFT step performs a homomorphic discrete Fourier transform, reinstating the ciphertext's capability for homomorphic operations. In this work, we will provide how to adapt our design to the bootstrapping procedure.

## 2.4 Approaches to Key-Switching

HMult and HRot generate intermediate ciphertexts with modified decryption keys, specifically $s^2$ and $\phi_r(s)$, respectively. Key-switching enables the transformation of ciphertexts, rendering them decryptable using the original secret key $s$.

**Hybrid key-switching (HKS):** This method [7, 29] starts by decomposing the input polynomial $[a]_Q \in \mathcal{R}_Q$ into dnum *digits*: $\{[a]_{\tilde{Q}_k}\}_{0 \leq k < \mathtt{dnum}}$, where $Q = \prod_{k=0}^{\mathtt{dnum}-1} \tilde{Q}_k$ and $\tilde{Q}_k = \prod_{i=\alpha k}^{\alpha(k+1)-1} q_i$. The evaluation key **evk** required for HKS comprises pairs of polynomials, specifically $\mathbf{evk} = \{\mathrm{evk}_0, \mathrm{evk}_1\} \in \mathcal{R}_{QP}^{2 \times \mathtt{dnum}}$, where $\mathrm{evk}_\tau = \{\mathrm{evk}_\tau^{(0)}, \mathrm{evk}_\tau^{(1)}, \cdots, \mathrm{evk}_\tau^{(\mathtt{dnum}-1)}\}$ for $\tau = 0, 1$. $P$ is an *auxiliary modulus* used for noise reduction, comprising $\alpha$ prime numbers.

The $k$-th entry $\{\mathrm{evk}_0^{(k)}, \mathrm{evk}_1^{(k)}\}$ of **evk** [7, 29] is generated as

$$\left\{ \left[ -a_k \cdot s + s' \cdot P \cdot [\hat{Q}_k^{-1}]_{\tilde{Q}_k} \cdot \hat{Q}_k + e_k \right]_{QP}, [a_k]_{QP} \right\} \in \mathcal{R}_{QP}^2, \tag{8}$$

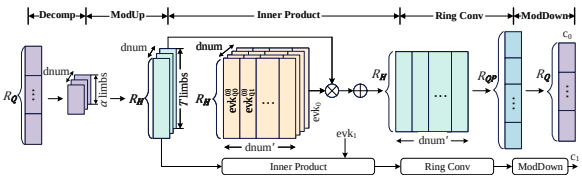



**Figure 2: Computational workflow of the KDS method. The generation of $c_1$ follows the same procedure as that of $c_0$.**

where $a_k$ is a random polynomial, $e_k$ is a small error polynomial, $\hat{Q}_k = Q/\tilde{Q}_k$, $0 \le k <$ dnum. Here, $s'$ is $\phi_r(s)$ / $s^2$ for HRot / HMult.

Fig. 1 depicts the computational workflow of the HKS method. The process primarily encompasses four sequential steps: Decomp, ModUp, Inner Product, and ModDown. Each step comprises multiple fundamental computational operations, typically involving expensive (I)NTT and BConv. Prior to computing the inner product, each digit $[a]_{\tilde{Q}_k}$ is first extended to $\mathcal{R}_{QP}$. The inner product step entails multiplying ciphertexts with the evaluation key, followed by an accumulation over $\mathcal{R}_{QP}$. Finally, ModDown is performed to bring the key-switching result back to $\mathcal{R}_Q$.

**Key decomposition switching (KDS):** This approach [36] further decomposes $\text{evk}_\tau^{(k)}$ into dnum′ slices: $\{[\text{evk}_\tau^{(k)}]_{Q_j}\}_{0 \le j <$ dnum′, where $QP = \prod_{j=0}^{\text{dnum}'-1} Q_j$, modulus $Q_j = \prod_{v=\alpha' j}^{\alpha'(j+1)-1} q_v$, dnum′ $= \lceil (L+\alpha)/\alpha' \rceil$, $\tau = 0, 1$, and $k = 0, 1, \cdots,$ dnum $- 1$. Based on the decomposition property [36], the $Q_j$-limb of inner product, $[A_{\text{InP}}]_{Q_j}$, can be formulated as:

$$[A_{\text{InP}}]_{Q_j} = \sum_{k=0}^{\text{dnum}-1} [a]_{\tilde{Q}_k} \cdot [\text{evk}_\tau^{(k)}]_{Q_j} \bmod Q_j \tag{9}$$

where $[a]_{\tilde{Q}_k}$ denotes the $k$-th decomposed digit as in HKS. Given that $A_{\text{InP},Q_j} = \sum_{k=0}^{\text{dnum}-1} [a]_{\tilde{Q}_k} \cdot [\text{evk}_\tau^{(k)}]_{Q_j}$ is bounded by

$$A_{\text{InP},Q_j} \le B_d = \text{dnum} \cdot N \cdot max\{\tilde{Q}_k\}/2 \cdot max\{Q_j\}/2 \tag{10}$$

this method introduces the *converted modulus* $H = \prod_{i=0}^{T-1} h_i > 2B_d$ that satisfies $H/2 > B_d$ to accommodate $A_{\text{InP},Q_j}$. Then, each $[A_{\text{InP}}]_{Q_j}$ can be computed as:

$$\begin{aligned} [A_{\text{InP}}]_{Q_j} &= \left(A_{\text{InP},Q_j}\right) \bmod Q_j \\ &= \left[ \sum_{k=0}^{\text{dnum}-1} \left[ [a]_{\tilde{Q}_k} \right]_H \cdot \left[ [\text{evk}_\tau^{(k)}]_{Q_j} \right]_H \right]_H \bmod Q_j \end{aligned} \tag{11}$$

Fig. 2 illustrates the computational diagram of the KDS method. The entire computation comprises five key steps: Decomp, ModUp, Inner Product, Ring Conv, and ModDown. Specifically, each decomposed digit is initially extended to the ring $\mathcal{R}_H$ through the ModUp computation. This is followed by the Inner Product step, which computes the inner product between the ciphertext and the evaluation key within $\mathcal{R}_H$. The outcome is then converted back to the ring $\mathcal{R}_{QP}$ via ring conversion (Ring Conv). In the end, ModDown is applied to produce the final key-switching output.

The primary distinction between the HKS and KDS methods lies in their inner product computation strategies. The HKS method performs the inner product directly on the ring $\mathcal{R}_{QP}$, whereas the KDS method employs a temporary ring, $\mathcal{R}_H$, for this purpose. By decomposing the evaluation key, the KDS method leverages a converted modulus $H$ that is substantially smaller than $QP$ [36]. This modulus size reduction yields a significant computational advantage, markedly decreasing the number of computationally expensive NTT operations during the key-switching process. From the theoretical perspective of asymptotic complexity, the HKS method necessitates $O(\ell^2)$ NTT operations, where $\ell$ denotes the ciphertext level. In contrast, the KDS method reduces the asymptotic complexity of the NTT computational load from quadratic to linear, achieving $O(\ell)$ with respect to the ciphertext level [36].

## 3 Motivation and Challenges

Key-switching encompasses multiple (I)NTTs and BConvs, as well as considerable memory resources for a large-sized evaluation key. Due to its frequent invocation, key-switching dominates the performance of HE applications [32, 46].

Over the years, several key-switching techniques have been developed. The BV method [6, 9] provides a larger level budget but sacrifices computational efficiency. The GHS approach [23] improves efficiency and mitigates noise growth, at the expense of halving the level budget. To strike a balance between efficiency and level budget, the HKS method was subsequently developed [29].

Regarding HE accelerator designs, F1 [45] pioneered the adoption of the BV method and developed the first HE accelerator based on this technique. Craterlake [46] conducted an in-depth analysis of the BV and HKS methods, focusing on their computational complexity and memory footprint. This study then centered on the HKS approach to advance hardware architecture design. Later efforts, such as BTS [37], FAB [2], ARK [35], and SHARP [34], embraced this switching methodology, progressively refining hardware designs to optimize computational efficiency.

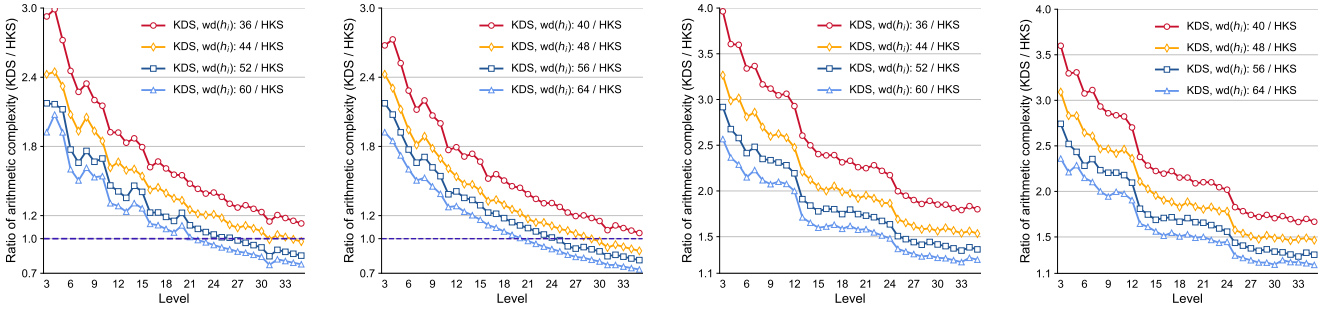
The KDS method [36], proposed in 2023, exhibits superior asymptotic complexity compared to the HKS approach. However, this computational advantage remains primarily theoretical at present. To date, there has been no related analysis of its practical efficacy, nor its potential impact on accelerator design. Below we present an analysis of the KDS method, focusing on computational complexity (measured by modular multiplications involved) and evaluation key size. This analysis is conducted based on the Lattigo library [41].

### 3.1 Increasing Computational Overhead at Short Word Length

The entire computation of the HKS method takes place within $\mathcal{R}_{QP}$. In contrast, as shown in Fig. 2, the KDS approach computes the inner products in the converted ring $\mathcal{R}_H$ and finally converts the results back to $\mathcal{R}_{QP}$. The converted modulus $H$ is composed of $T$ individual prime numbers $h_i$, i.e., $H = \prod_{i=0}^{T-1} h_i$.

We observe that the computational complexity of the KDS method is closely tied to the word length choice of $h_i$. Specifically, a larger word length for $h_i$ results in improved computational efficiency of the KDS method. In Fig. 3, we present the complexity comparisons of the KDS and HKS methods with the parameter sets ($N = 2^{16}, L = 35, \log q_i = 36, \alpha = 5, 12$). These comparisons evaluate the KDS method across a range of word lengths for $h_i$ (wd($h_i$) for short), ranging from 36 to 64 bits.

As wd($h_i$) increases, the computing cost of the KDS method gradually decreases. When $\alpha = 5$(dnum $= L/\alpha = 7$), wd($h_i$) needs to be




**(a)** $\alpha = 5$, $\mathtt{wd}(h_i) = 36, 44, 52, 60$. **(b)** $\alpha = 5$, $\mathtt{wd}(h_i) = 40, 48, 56, 64$. **(c)** $\alpha = 12$, $\mathtt{wd}(h_i) = 36, 44, 52, 60$. **(d)** $\alpha = 12$, $\mathtt{wd}(h_i) = 40, 48, 56, 64$.

**Figure 3: Complexity comparisons of the KDS and HKS methods in terms of modular multiplications (mod mults). The ratio is computed as the minimum number of mod mults in KDS divided by the mod mult number in HKS. The word length $\mathtt{wd}(h_i)$ utilized in the KDS method ranges from 36 to 64 bits.**

at least 48 bits (Fig. 3(b)) for the KDS method to gain a computational advantage over HKS at several of the higher levels ($\ell$ from 31 to 35). Conversely, when $\mathtt{wd}(h_i)$ is set to a shorter word length, specifically less than 48 bits, HKS consistently outperforms KDS across all levels. Particularly, when $\mathtt{wd}(h_i) = \log q_i = 36$ (Fig. 3(a)), KDS is 1.13× to 2.99× computationally more expensive than HKS. Even when $\mathtt{wd}(h_i)$ reaches 64 bits for the KDS method (Fig. 3(b)), the HKS approach still outperforms at levels lower than 21.

As $\mathtt{dnum}$ decreases, the disparity in computing overhead between the two switching methods becomes more pronounced. When $\alpha = 12$ ($\mathtt{dnum} = 3$), even with $\mathtt{wd}(h_i)$ increased to 64 bits (Fig. 3(d)), the KDS method remains significantly inferior to the HKS approach across all levels, incurring at least a 1.19× higher computing cost ($L = 35$). Furthermore, when $\mathtt{wd}(h_i)$ is set to shorter word lengths, such as 44, 40, and 36 bits, the computational overhead of the KDS method grows rapidly. In these cases, the KDS approach is at least 1.54×, 1.66×, and 1.8× computationally more expensive than the HKS method at the high levels, respectively.

## 3.2 Incompatible with Fixed-Word Accelerators

The machine word length critically influences the hardware cost of FHE accelerators. Recent research [34] demonstrates that a relatively compact 36-bit word length is sufficient to robustly support FHE workloads. Conversely, as analyzed in Sec. 3.1, the word length of $h_i$ for the KDS method needs to be set to a large size (e.g., 64 bits) to effectively reduce computing cost. The divergent word-length requirements render the KDS method incompatible with the fixed-word architecture designs used in previous HE accelerators.

The arithmetic logic units (ALUs) in HE implementations primarily consist of general multipliers and modular multipliers. The hardware costs of these ALUs exhibit a near-quadratic relationship with the word length. For example, the 64-bit multiplier (modular multiplier) occupies 2.75× (3.28×) more area and consumes 2.83× (3.31×) more power compared to its 36-bit counterpart.

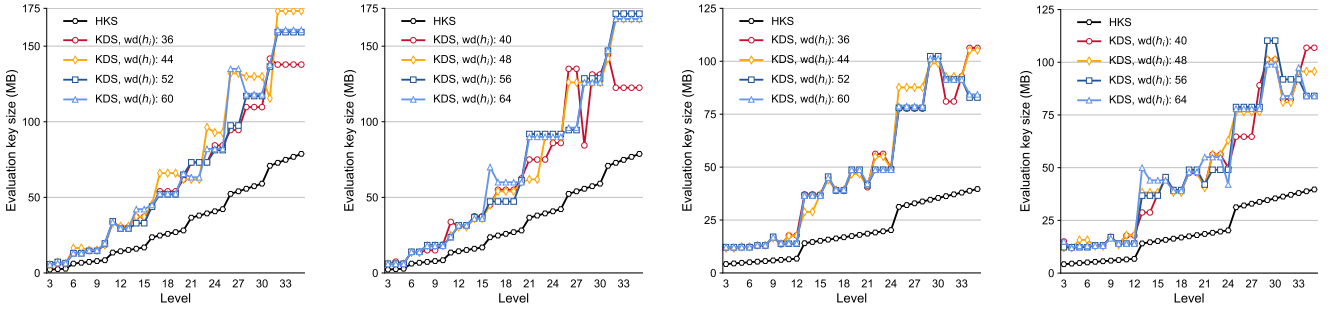
Thus, **(i)** if the accelerator word length is set directly to the longer $\mathtt{wd}(h_i)$, the hardware utilization for HE operations with the smaller $\log q_i$-bit word length will significantly decrease, only apart from the KDS process utilized at several high levels. **(ii)** Alternatively, one can break down operations at various word lengths into smaller base arithmetic primitives, and leverage the configuration and combination of base functional units to support operations at both word lengths. For instance, we can utilize the 18-bit multiplier as the base primitive, and perform the 36-bit and 64-bit multiplications leveraging multiple 18-bit multipliers. This approach, however, leads to complex run-time reconfiguration of functional units, along with substantial control and wiring overhead. In addition, such a design may still lead to inefficiencies in utilization. More importantly, regardless of the design choices, the hardware overhead of functional units will inevitably escalate due to the necessity of supporting long-width modular arithmetic operations for the KDS method.

## 3.3 Inflation of Memory Requirements

The evaluation key (**evk**) in key-switching occupies a substantial memory footprint, particularly at high levels. In the KDS method, each decomposed $[\mathbf{evk}_i]_{Q_j}$ is expanded into the converted ring $\mathcal{R}_H$ ($Q_j << H$), inevitably leading to an increase in key size.

Fig. 4 presents evk sizes of the KDS and HKS methods with the parameter sets ($N = 2^{16}, L = 35, \log q_i = 36, \alpha = 5, 12$). When $\alpha = 5$, the **evk** of KDS ($\mathbf{evk}_{\mathrm{KDS}}$) grows rapidly by 1.5× to 3× times compared to the **evk** of HKS ($\mathbf{evk}_{\mathrm{HKS}}$). At the highest level $L = 35$, $\mathbf{evk}_{\mathrm{KDS}}$ expands by a factor of 1.56× (Fig. 4(b), $\mathtt{wd}(h_i) = 40$) to 2.2× (Fig. 4(a), $\mathtt{wd}(h_i) = 44$). As the number of decomposition digits $\mathtt{dnum}$ decreases, the $\mathbf{evk}_{\mathrm{KDS}}$ inflation issue becomes more pronounced. As shown in Fig. 4(c)-(d), when $\alpha = 12 (\mathtt{dnum} = 3)$, $\mathbf{evk}_{\mathrm{KDS}}$ experiences a dramatic growth of 1.97× to 3.56× than $\mathbf{evk}_{\mathrm{HKS}}$. In this case, at the highest level, $\mathbf{evk}_{\mathrm{KDS}}$ expands by 2.1× (Fig. 4(c), $\mathtt{wd}(h_i) = 52$) to 2.7× (Fig. 4(d), $\mathtt{wd}(h_i) = 40$) across distinct $\mathtt{wd}(h_i)$ values.

It is noteworthy that, unlike its computational overhead, the $\mathbf{evk}_{\mathrm{KDS}}$ size in key decomposition switching does not diminish as the growth of $\mathtt{wd}(h_i)$. The swift expansion of $\mathbf{evk}_{\mathrm{KDS}}$, particularly at high levels, significantly exacerbates the memory bottleneck in HE accelerators. One option is to offload part of the **evk**s to off-chip memory and then load them on the chip as needed. However, in this scenario, the load time of such large **evk**s is the critical factor dominating the HE accelerator performance [34, 35, 37]. An alternative approach is to increase the on-chip memory capacity substantially to accommodate the expanded **evk**s. However, this option comes with a significant increase in hardware costs.




**(a)** $\alpha = 5$, $\mathtt{wd}(h_i) = 36, 44, 52, 60$. **(b)** $\alpha = 5$, $\mathtt{wd}(h_i) = 40, 48, 56, 64$. **(c)** $\alpha = 12$, $\mathtt{wd}(h_i) = 36, 44, 52, 60$. **(d)** $\alpha = 12$, $\mathtt{wd}(h_i) = 40, 48, 56, 64$.

**Figure 4: Evk sizes required for the HKS and KDS methods when using the pseudo random number generator to generate half of the evaluation key. The word length $\mathtt{wd}(h_i)$ utilized in the KDS method ranges from 36 to 64 bits.**

### 3.4 Hardware-Unfriendly Rounding Operation

In the KDS method, an exact base conversion ExactBConv is required for the ring conversion from $\mathcal{R}_H$ to $\mathcal{R}_{Q_j}$ [24, 36].

$$\begin{aligned}&\mathtt{ExactBConv}\left([A]_H, H{\rightarrow}Q_j\right)\\&= \mathtt{BConv}\left([A]_H, H{\rightarrow}Q_j\right) - z\cdot H \bmod q_v, 0 \le j < \mathtt{dnum}'\end{aligned} \tag{12}$$

$$z = \left\lfloor \frac{[A]_H + z\cdot H}{H} \right\rceil = \left\lfloor \sum_{i=0}^{\alpha'-1} \left[ [A]_{h_i} \cdot \hat{h}_i^{-1} \right]_{h_i} / h_i \right\rceil \tag{13}$$

Compared to BConv in HKS, ExactBConv incorporates an additional subtraction term, $[z\cdot H]_{q_j}$. The value of $z$ is obtained through division, accumulation, and rounding. To circumvent high-precision division, $1/h_i$ is represented as a high-precision decimal, which is then multiplied by $[[A]_{h_i}\cdot \hat{h}_i^{-1}]_{h_i}$ to complete the division [24]. To ensure that the final rounding error is bounded by $2^{-\epsilon}$, the precision of $1/h_i$ should be at least $2^{-\zeta}$, where $\zeta = \epsilon + \log max\{h_i\} + \log \alpha'$. For instance, to achieve a rounding error less than $2^{-72}$, the precision of $1/h_i$ needs to be at least $2^{-144}$ when $\alpha' = 35$, $\log h_i = 36$.

For hardware, this rounding operation is particularly expensive. In hardware implementation, $1/h_i$ is represented as a number with an exceptionally large bit width, specifically $\zeta$ bits. Consequently, executing this rounding operation in hardware necessitates high-bit-width multipliers and large-bit-width adders. To support the KDS, additional expensive circuit costs are introduced to the hardware accelerators. Despite these costly investments, rounding errors persist, potentially compromising the reliability.

## 4 Method of Fixed-Word Key Decomposition Switching (FW-KDS)

The KDS method involves two distinct types of word length operations: those belonging to $\mathcal{R}_{QP}$ and those within $\mathcal{R}_H$. This method only offers a computational advantage when employing a large word length $\mathtt{wd}(h_i)$ for operations within $\mathcal{R}_H$ (Sec. 3.1). This results in incompatibility with previous fixed-word-size FHE accelerators. Intuitively, two approaches can be employed to facilitate KDS in accelerator design. The first is a straightforward extension of the machine word length to match $\mathtt{wd}(h_i)$. The second way entails breaking down operations under both word lengths into finer arithmetic granules and leveraging the combination of functional granules to handle operations at both word lengths. The drawbacks of both have been detailed in Sec. 3.2. Importantly, neither design approach addresses the explosive memory demands of evaluation keys introduced by the KDS method.

### 4.1 Fixed-Word Modulus Selection for $\mathcal{R}_H$

We propose constructing the RNS basis of converted modulus $H$ by selecting $h_i$ from the existing modulus set $\mathcal{D} = \{q_i\}_{0\le i<L+\alpha}$ of $QP$. Given that $2B_d < H < QP$ (Sec. 2.4), there must exist a modulus subset $\tilde{\mathcal{D}} = \{q_i\}_{u\le i<r} \subsetneq \mathcal{D}$ fulfilling $\prod_{i=u}^{r-1} q_i > 2B_d$. For instance, we can directly select the moduli of $max\{\tilde{Q}_i\}$ and $max\{Q'_j\}$, along with any additional modulus $q_k (>> \mathtt{dnum}\cdot N/2)$. This selection clearly ensures compliance with the stipulated inequality.

Through such a selection strategy for the RNS basis of modulus $H$, we can transform arithmetic operations within $\mathcal{R}_H$, as well as the base conversions between $\mathcal{R}_H$ and $\mathcal{R}_{QP}$, into the $\mathtt{wd}(q_i)$-bit operations. This approach unifies the machine word length of an FHE accelerator to a fixed size of $\mathtt{wd}(q_i)$ when employing the key decomposition switching. However, as detailed in Sec. 3.1, in the case of $\mathtt{wd}(h_i) = \mathtt{wd}(q_i)$, the computational cost of key decomposition switching escalates significantly compared to hybrid key-switching. We subsequently delve into strategies to mitigate the computational complexity associated with this fixed word length.

### 4.2 Algorithmic Optimizations

With the specified modulus selection, let us denote the converted modulus $H$ greater than $2B_d$ as $H = \prod_{i=0}^{T-1} q_i$. For each $q_{0\le i<T}$, the value $A_{\mathrm{InP},q_i} = \sum_{k=0}^{\mathtt{dnum}-1} [a]_{\tilde{Q}_k} \cdot [\mathrm{evk}_\tau^{(k)}]_{q_i}$ is bounded by $A_{\mathrm{InP},q_i} < \mathtt{dnum}\cdot N\cdot max\{\tilde{Q}_k\}/2\cdot max\{q_i\}/2$. Since $max\{q_i\} \le Q_j$ (equal when $\alpha' = 1$), we still have $A_{\mathrm{InP},q_i} \le B_d < H/2$ (Sec. 2.4). This indicates that the converted modulus $H$ can accommodate $A_{\mathrm{InP},q_i}$. Thus, based on Eq. 11, the $q_i$-limb inner product can be still computed as:

$$\begin{aligned}[A_{\mathrm{InP}}]_{q_i} &= \left[ \sum_{k=0}^{\mathtt{dnum}-1} \left[ [a]_{\tilde{Q}_k} \right]_H \cdot \left[ [\mathrm{evk}_\tau^{(k)}]_{q_i} \right]_H \right]_H \bmod q_i\\ &= \left\{ \sum_{k=0}^{\mathtt{dnum}-1} \left[ [a]_{\tilde{Q}_k} \right]_H \cdot \left[ [\mathrm{evk}_\tau^{(k)}]_{q_i} \right]_H - d\cdot H \right\} \bmod q_i\\ &= \sum_{k=0}^{\mathtt{dnum}-1} \left[ [a]_{\tilde{Q}_k} \right]_{q_i} \cdot [\mathrm{evk}_\tau^{(k)}]_{q_i} \bmod q_i\end{aligned} \tag{14}$$

The last equality holds because $q_i$ is a factor of $H$. Eq. 14 demonstrates that for each $q_i \in \mathcal{H} = \{q_i\}_{0\le i<T}$, the $q_i$-limb computation

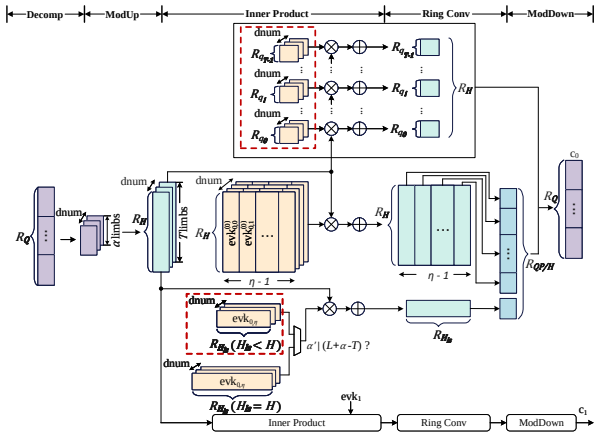



**Figure 5: Computational workflow of the FW-KDS method. When $\alpha'$ exactly divides $(L+\alpha-T)$, the last converted modulus $H_{ls}$ is identical to $H$, and $\mathsf{evk}_{0,\eta}$ resides in $\mathcal{R}_H$; in all other cases, $H_{ls}$ is smaller than $H$, and $\mathsf{evk}_{0,\eta}$ belongs to $\mathcal{R}_{H_{ls}}$. Evaluation keys highlighted in red dashed boxes indicate those size-reduced components compared to KDS.**

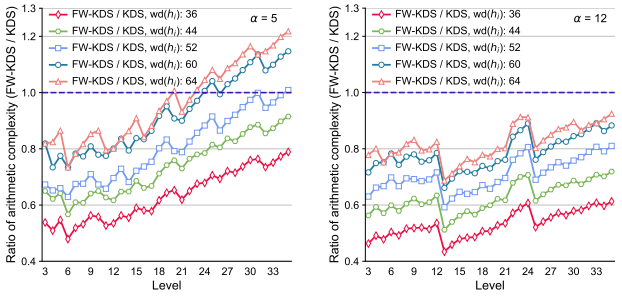
$[A_{InP}]_{q_i}$ can be performed directly over the subring $\mathcal{R}_{q_i}$. This approach eliminates the necessity to operate over the entire ring $\mathcal{R}_H$, thereby reducing computational complexity, as $q_i$ is significantly smaller than $H$. The computation highlighted within the black box in Fig. 5 corresponds to this algorithmic optimization.

For the remaining modulus set, $\mathcal{D}\backslash\mathcal{H} = \{q_i\}_{T\le i<L+\alpha}$, we still follow the original KDS method to derive each $Q_j$-limb inner product $[A_{\text{InP}}]_{Q_j}$, where $QP/H = \prod_{j=0}^{\eta-1} Q_j$ and $\eta = \lceil (L+\alpha-T)/\alpha' \rceil$. Consider the last modulus $Q_{\eta-1}$ in the modulus set $\mathcal{D}\backslash\mathcal{H}$. Often, $Q_{\eta-1}$ comprises fewer than $\alpha'$ prime moduli when $(L+\alpha-T)$ is not divisible by $\alpha'$. In such cases, the upper bound of $A_{\text{InP},Q_{\eta-1}} = \sum_{k=0}^{\mathsf{dnum}-1} [a]_{\tilde{Q}_k} \cdot [\mathsf{evk}_i^{(k)}]_{Q_{\eta-1}}$ is much smaller than $H$. This implies that we can utilize a smaller converted modulus $H_{ls}$ to accommodate the value of $A_{\text{InP},Q_{\eta-1}}$. To efficiently leverage existing NTT results of $[[a]_{\tilde{Q}_k}]_H$ over $\mathcal{R}_H$, we can strategically select $H_{ls}$ as a divisor of $H$, specifically, $H_{ls}/2 = \prod_{i=0}^{t-1} q_i/2 > A_{\text{InP},Q_{\eta-1}}$ with $t < T$. Consequently, we can perform the computation of $[A_{\text{InP}}]_{Q_{\eta-1}}$ (Fig. 5) on the smaller ring $\mathcal{R}_{H_{ls}}$ instead of $\mathcal{R}_H$ (Eq. 11), thereby mitigating computational overhead. When $(L+\alpha-T)$ is divisible by $\alpha'$, the last converted modulus $H_{ls}$ is simply identical to $H$.

Fig. 5 shows the computational workflow of the modified KDS method after incorporating our optimizations. It is named as *Fixed-Word Key Decomposition Switching* (FW-KDS), reflecting the uniform word length applied in both the rings $\mathcal{R}_{QP}$ and $\mathcal{R}_H$.

## 4.3 Reduction of Computational Overhead

In the KDS method (see Fig. 2), for each $\mathcal{R}_{Q_j}$ with $0 \le j < \mathsf{dnum}'$, the inner products modulo $Q_j$ are first performed over $\mathcal{R}_H$, demanding $2 \cdot \mathsf{dnum} \cdot T \cdot N$ multiplications. Subsequently, Ring Conv is applied to convert the computational results from $\mathcal{R}_H$ back to $\mathcal{R}_{Q_j}$. To facilitate this ring conversion, the inner product results need to be first converted from the evaluation representation back to the coefficient representation, requiring $2 \cdot T$ INTT operations. Finally,




**(a)** $\alpha = 5$, $\mathtt{wd}(h_i) = 36, 44, 52, 60, 64$. **(b)** $\alpha = 12$, $\mathtt{wd}(h_i) = 36, 44, 52, 60, 64$.

**Figure 6: Complexity comparisons of the FW-KDS and KDS methods. Ratio is computed as the minimum number of mod mults in FW-KDS divided by the minimum mod mult number in KDS. Word length for FW-KDS is 36 bits. $\mathtt{wd}(h_i)$ denotes the word length of converted modulus $h_i$ utilized in KDS.**

the computational results over ring $\mathcal{R}_{Q_j}$ are transformed back into the evaluation representation, involving $2 \cdot \alpha'$ NTT operations.

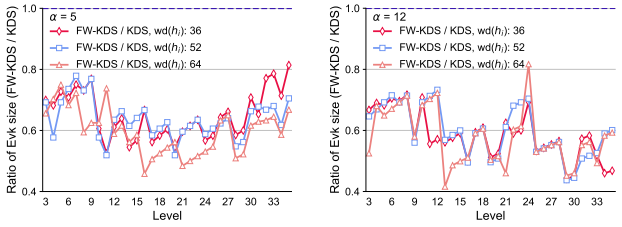
The FW-KDS approach partitions the inner product computation into three parallel components: **(i)** For each $Q_j \in \mathcal{H}$, the inner products are performed directly on each $q_i$, reducing the required multiplications from $2 \cdot \mathsf{dnum} \cdot T \cdot N$ to $2 \cdot \mathsf{dnum} \cdot \alpha' \cdot N$. Since the computation results consistently reside within the subring $\mathcal{R}_{Q_j=\prod q_i}$, the Ring Convs from $\mathcal{R}_H$ to $\mathcal{R}_{Q_j}$, along with the accompanying NTTs and INTTs, are completely eliminated. **(ii)** The second part focuses on the computation for the remaining moduli $Q_j$ (excluding $Q_{\eta-1}$), following the same procedure as in the KDS method. **(iii)** The third component addresses the computation for the last modulus $Q_{\eta-1}$, which is performed over smaller $\mathcal{R}_{H_{ls}}$. This step reduces the required multiplications from $2 \cdot \mathsf{dnum} \cdot T \cdot N$ to $2 \cdot \mathsf{dnum} \cdot t \cdot N$, while also decreasing the expensive INTTs from $2T$ to $2t$.

The choice of $\alpha'$, ranging from 1 to $(\ell+\alpha-1)$, affects the computational complexity. For each level, we can obtain the minimal computational overhead of key-switching by exhaustively searching through all possible $\alpha'$ values. Fig. 6 presents complexity comparisons between the FW-KDS and KDS methods, both achieving their respective minimum number of mod mults at each level.

For $\alpha = 5$, FW-KDS exhibits a reduction ranging from 52.1% to 20.4% compared to the KDS approach at the equivalent word length ($\mathtt{wd}(h_i) = 36$, Fig. 6(a)). Remarkably, this efficiency persists even when the KDS word length is extended to 64 bits, whilst FW-KDS operates at 36 bits, decreasing computational costs by up to 26.6% for levels below 20. Upon increasing $\alpha$ to 12, the FW-KDS method achieves an even greater diminution in computational overhead. In this case, even if the word length used for KDS is increased to 64 bits, the FW-KDS method consistently maintains a lower computing cost across all levels, yielding computational savings of up to 30.4%. At equivalent word lengths, FW-KDS demonstrates a remarkable 56% ~ 39.1% reduction in computational overhead across all levels.

## 4.4 Reduction of Memory Requirements

In HE accelerators, the **evk** for key-switching consumes a substantial fraction of on-chip memory resources, particularly at high ciphertext levels [34, 35]. This memory allocation strategy avoids fetching **evk** data from off-chip memory, thus alleviating the impact




**(a)** $\alpha = 5$, $\texttt{wd}(h_i) = 36, 52, 64$. **(b)** $\alpha = 12$, $\texttt{wd}(h_i) = 36, 52, 64$.

**Figure 7: Evk size comparisons of FW-KDS and KDS. The ratio is computed as the evk size in FW-KDS divided by that in KDS. FW-KDS utilizes a 36-bit word length. $\texttt{wd}(h_i)$ denotes the word length of the converted modulus $h_i$ in KDS.**

of bandwidth bottlenecks. In the KDS method, each slice $[\text{evk}_\tau^{(k)}]_{Q_j}$ undergoes an expansion to $[[\text{evk}_\tau^{(k)}]_{Q_j}]_H$ over $\mathcal{R}_H$ (Sec. 2.4). The latter consists of $2T$ polynomial limbs. Consequently, the total size of **evk** for KDS amounts to $2 \cdot \texttt{dnum} \cdot \texttt{dnum}' \cdot T$ limbs.

As shown in Fig. 5, the **evk** for FW-KDS consists of three components: **(i)** For each $q_i \in \mathcal{H} = \{q_i\}_{0 \le i < T}$, the key for inner products remains as $[\text{evk}_\tau^{(k)}]_{q_i}$ (Sec. 4.2, Eq. 14), without requiring any additional expansion. Each term $[\text{evk}_\tau^{(k)}]_{q_i}$ follows the generation formulated in Eq. 8 modulo prime $q_i$, specifically $[\text{evk}_\tau^{(k)}]_{QP} \bmod q_i$. Hence, the size of this component is $2T$ limbs. In contrast, in the original KDS method, these keys undergo an expansion (Eq. 11), resulting in a total of $2 \cdot \lceil T/\alpha' \rceil \cdot T$ limbs. **(ii)** For the remaining moduli $Q_j$ (excluding $Q_{\eta-1}$), each slice $[\text{evk}_\tau^{(k)}]_{Q_j} \in \mathcal{R}_{Q_j}$ is expanded into $\mathcal{R}_H$, consistent with the KDS method. This component can be formulated as $[[\text{evk}_\tau^{(k)}]_{Q_j}]_H$, which follows $\texttt{ExactBConv}([\text{evk}_\tau^{(k)}]_{Q_j}, Q_j \to H)$ (Eq. 12). **(iii)** For the last modulus $Q_{\eta-1}$, the inner product is performed over the smaller ring $\mathcal{R}_{H_{ls}}$. In this case, the required key is reduced from $[[\text{evk}_\tau^{(k)}]_{Q_{\eta-1}}]_H$ in KDS to $[[\text{evk}_\tau^{(k)}]_{Q_{\eta-1}}]_{H_{ls}}$, which can be further formulated as $\texttt{ExactBConv}([\text{evk}_\tau^{(k)}]_{Q_{\eta-1}}, Q_{\eta-1} \to H_{ls})$ (Eq. 12). Therefore, a size of $2 \cdot \texttt{dnum} \cdot (T - t)$ limbs is decreased at this step.

Fig. 7 presents the comparisons of the **evk** sizes required by FW-KDS and KDS when achieving their respective minimal computational overhead (Fig. 6). The choice of decomposition parameter $\alpha'$ at each level is set to minimize method computational complexity through exhaustively searching as described in Sec. 4.3. When $\alpha = 12$, the **evk** size in FW-KDS is reduced by up to 58.4% across all levels compared to KDS with varying word lengths. Similarly, for $\alpha = 5$, adopting the FW-KDS method results in a reduction of up to 54.2% in **evk** size. Furthermore, at high levels, the reduction in **evk** size using FW-KDS ranges from 49.1% to 18.6% when $\alpha = 5$. As $\alpha$ increases, the FW-KDS method demonstrates more effective reductions in **evk** size at high levels. Specifically, for $\alpha = 12$, the **evk** size reduction ranges from 56.3% to 39.9% at high levels.

## 5 Applying FW-KDS for Hardware Acceleration

In this section, we elucidate the integration of the FW-KDS method into accelerator designs with minimal storage overhead, thereby enhancing computational efficiency at both high and medium levels.

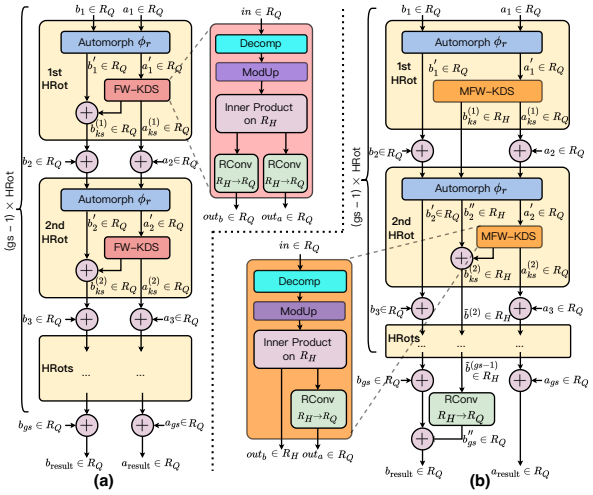



**Figure 8: (a) Computational flow of the H-IDFT giant step utilizing the minimum key-switching strategy. Ring conversion (RConv) $\mathcal{R}_H \to \mathcal{R}_Q$ denotes the process of performing exact base conversion from $\mathcal{R}_H$ to $\mathcal{R}_{QP}$, and then conducting ModDown from $\mathcal{R}_{QP}$ back to $\mathcal{R}_Q$. (b) Computational flow of the H-IDFT giant step adopting the proposed Half-RConv strategy. A total of $(\texttt{gs} - 2)$ RConvs are reduced. The modified FW-KDS combined with Half-RConv is termed MFW-KDS.**

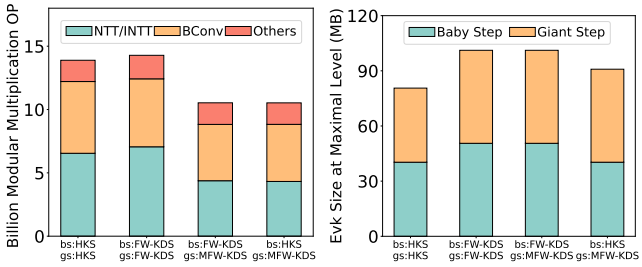
### 5.1 Applying to H-IDFT at High Levels

The minimum key-switching approach [25, 35] is proposed to greatly enhance the reusability of **evk**s. Through this, the BSGS algorithm for each H-IDFT iteration requires only two **evk**s.

Fig. 8(a) depicts the computational flow of the giant step process in H-IDFT when adopting the minimum key-switching strategy. The entire process consists of $\texttt{gs} - 1$ consecutive sub-steps, each corresponding to a single HRot operation. All HRot operations share the same rotation amount $r$, and thus utilize the same single evaluation key $\textbf{evk}_{\text{rot}}^{(r)}$ for the key-switching computation.

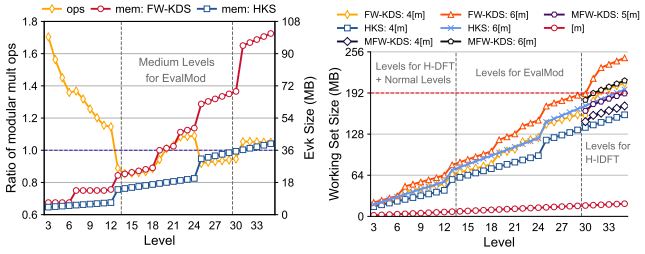
In the initial sub-step, we apply HRot to the input ciphertext $c_1 = (b_1, a_1) \in \mathcal{R}_Q^2$. Specifically, $(b_1, a_1)$ first undergoes an automorphism $\phi_r$ to produce $(b_1', a_1') = (\phi_r(b_1), \phi_r(a_1))$. Then, the second component $a_1'$ undergoes the key-switching routine of FW-KDS, yielding $(b_{ks}^{(1)}, a_{ks}^{(1)})$. This sub-step concludes with the addition $b_1' + b_{ks}^{(1)}$. The second step involves adding the input $c_2 = (b_1, a_1) \in \mathcal{R}_Q^2$ to the result of the first sub-step, followed by performing the HRot operation. Subsequent sub-steps proceed in a similar iterative manner. For the sake of clarity, we take the initial and second sub-steps as illustrative examples in the following.

*5.1.1 Eliminate half of expensive ring conversions.* Applying the FW-KDS method to the H-IDFT giant step process (Fig. 8(a)) necessitates the ring conversion $\mathcal{R}_H \to \mathcal{R}_Q$ (abbreviated as RConv shown in Fig. 8) for each component of the key-switching result. Specifically, this ring conversion involves performing exact base conversion from $\mathcal{R}_H$ to $\mathcal{R}_{QP}$, and then executing ModDown from $\mathcal{R}_{QP}$ back to $\mathcal{R}_Q$. In the $i$-th sub-step, the output of FW-KDS $(b_{ks}^{(i)}, a_{ks}^{(i)})$




(a) Computation breakdown. (b) Evk breakdown.

**Figure 9: (a) Computation breakdown of H-IDFT for baby and giant steps using various methods. (b) Evk size breakdown for baby and giant steps using different methods.**




(a) FW-KDS vs HKS (b) Working set size

**Figure 10: (a) Comparison of FW-KDS and HKS methods. The ratio of operations is computed as FW-KDS/HKS. Notably FW-KDS targets minimizing the overall computational overhead at medium levels in this comparison. (b) Working set sizes under different key-switching methods when working on an evk and 4, 6 temporary ciphertexts (4[m], 6[m]) across various levels. [m] represents the size of a single ciphertext.**

is added component-wise to the automorphism result $b'_i$ and the next input $c_{i+1} = (b_{i+1}, a_{i+1})$. The cases of $i = 1, 2$ correspond to the initial and second sub-steps illustrated in Fig. 8(a). Since both $b'_i$ and $c_{i+1}$ reside in the ring $\mathcal{R}_Q$, the two output components of FW-KDS need to undergo the ring conversion $\mathcal{R}_H \rightarrow \mathcal{R}_Q$ to enable subsequent additions. This conversion involves multiple INTTs, `ExactBConv`s, and `BConv`s. Throughout the entire giant step process, this conversion is performed a total of $2 \cdot (\mathtt{gs} - 1)$ times.

We observe that the input $a'_2$ for the 2nd-sub-step FW-KDS is derived from the output $a^{(1)}_{ks}$ of the 1st-sub-step FW-KDS. As the input for key-switching is required to reside in $\mathcal{R}_Q$ to facilitate decomposition, the ring conversion $\mathcal{R}_H \rightarrow \mathcal{R}_Q$ is necessary for the second output component $a^{(1)}_{ks}$ of FW-KDS. However, for the first output component $b^{(1)}_{ks}$ of FW-KDS, only addition and automorphism operations are performed subsequently, and therefore, this component does not serve as the input for subsequent key-switching.

Based on this observation, we propose the *Half-RConv* strategy. As depicted in Fig. 8(b), we split the output ciphertext of the 1st sub-step into three components: $b'_1$, $b^{(1)}_{ks}$, and $a^{(1)}_{ks}$. In the 2nd sub-step, $b'_1$ and $a^{(1)}_{ks}$ are added to the $\mathcal{R}_Q$ inputs $b_2$ and $a_2$, respectively. The output component $b^{(2)}_{ks}$ of the 2nd-sub-step FW-KDS is then added to the rotated $b^{(1)}_{ks}$. In general, this partitioning strategy results in the following computational pattern: **(i)** the first output component of the $i$-th sub-step always originates from the entries $b_j \in \mathcal{R}_Q$ of the previous input $c_i$ for $j = 0, 1, \cdots, i-1$. **(ii)** Its second output component consistently stems from the 1st elements of the previous-sub-step FW-KDS results. **(iii)** The third output component is always the 2nd element $a^{(i)}_{ks}$ of current-sub-step FW-KDS results and undergoes FW-KDS during the $(i+1)$-th sub-step.

Particularly, the second output component of each sub-step is indeed the accumulation of the rotated 1st elements of previous FW-KDS results (e.g., $\tilde{b}^{(2)} = \psi_r(b^{(1)}_{ks}) + b^{(2)}_{ks}$ for the 2nd sub-step in Fig. 8(b)). This output component is completely irrespective of any element in $\mathcal{R}_Q$. Consequently, we can consistently maintain the first output item of FW-KDS in the ring $\mathcal{R}_H$, eliminating the need for the ring conversion $\mathcal{R}_H \rightarrow \mathcal{R}_Q$ at each sub-step.

After completing the $(\mathtt{gs} - 1)$ `HRot`s, we finally convert this component back from $\mathcal{R}_H$ to $\mathcal{R}_Q$. Hence, the second output component only requires a single ring conversion at the end of the process. As a result, the total number of ring conversions required for the entire giant step is reduced from $2 \cdot (\mathtt{gs} - 1)$ to gs, roughly eliminating half of the ring conversions. For brevity, we refer to the modified FW-KDS adopting Half-RConv as *MFW-KDS*.

*5.1.2 Combine two key-switching methods.* Fig. 9 presents the complexity comparisons of H-IDFT when its baby and giant steps employ varying key-switching approaches. Taking into account the on-chip memory capacity and the number of temporary ciphertexts required, the step size bs for the baby step is set to 4 in H-IDFT [34]. Similarly, for the FW-KDS method, the decomposition parameter $\alpha'$ is chosen to minimize the **evk** size. Specifically, $\alpha'$ is set to 21, 20, and 18 for levels $\ell = 35$, 33, and 31, respectively.

As illustrated in Fig. 9, employing the FW-KDS method for both the baby and giant steps results in a 2.81% increase in computational overhead compared to exclusively relying on HKS for both steps. Additionally, the computational costs for (I)NTT are 7.8% higher when FW-KDS is applied across both steps.

When both FW-KDS and Half-Conv are further applied during the giant step (baby step still uses FW-KDS), the overall computational cost is reduced by 26.3%. The computational expenses for (I)NTT and `BConv` decrease by 37.97% and 17.1%, respectively. This improvement is attributed to the elimination of nearly half of the ring conversions in the giant step, as detailed in Sec. 5.1.1. Furthermore, compared to exclusively using the HKS method in both steps, the overall computational overhead, along with the costs of NTT and BConv, decreases by 24.2%, 33.1%, and 21.5%, respectively. Moreover, in this case, further adopting HKS for baby step leads to a slight additional reduction in computational cost. This reduction is minimal, as the step size of baby step is only 4.

With respect to memory requirements, Fig. 9 also presents the **evk** sizes at the maximum level ($L = 35$) required for varying key-switching methods. Specifically, the **evk** size for the FW-KDS method reaches 50.6MB when utilizing pseudo random number generator (PRNG) to generate half of **evk** [46]. This size is 10.3MB larger than the **evk** required by the HKS approach. Taking both computational cost and **evk** size into account, we opt to apply HKS for the baby step and FW-KDS combined with Half-RConv

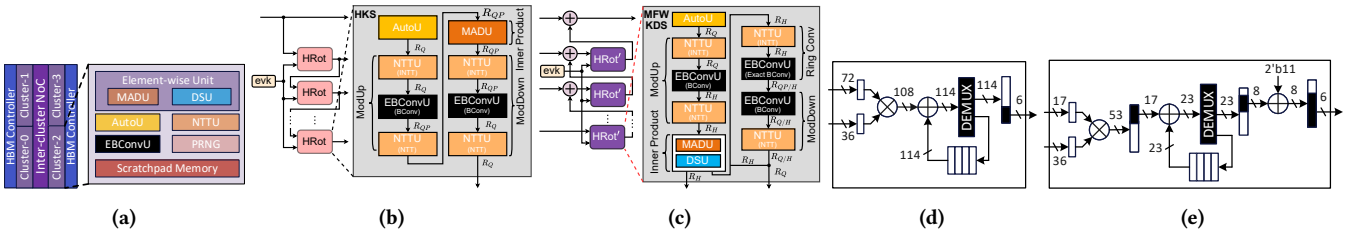



**Figure 11: (a) Overview of HAWK architecture. (b) Execution and data flow for baby step utilizing HKS in H-IDFT. The bracket within box indicates the operation performed by functional unit. (c) Execution and data flow for giant step adopting MFW-KDS in H-IDFT. (d) Rounding circuit using traditional computation method. (e) Rounding circuit employing proposed exact method.**

for the giant step. This strategic combination achieves the lowest computational overhead, reducing the overall computational cost to 75.78% of the conventional method that relies solely on HKS, while incurring only a 10.3 MB increase in **evk** size.

## 5.2 Applying FW-KDS to Medium Levels

The previous section demonstrates the application of the FW-KDS method at the highest, most computationally and memory-intensive levels. When $\mathtt{wd}(q_i) = 36$, H-IDFT consumed $3 \times 2$ levels, employing double-prime scaling. Thus the remaining levels range from $\ell = 29$ to 3. Fig. 10(b) shows the working set sizes across levels for distinct key-switching methods. This size represents the storage demand for a single `HMult`/`HRot` (both requiring key-switching), comprising an **evk** and varying temporary ciphertexts. MFW-KDS is only applicable to H-IDFT at high levels ($\ell = 35$ to 30) as detailed in Sec. 5.1. Moreover, MFW-KDS for high levels targets the minimum **evk** size, thus incurring only a small increase in key size (Sec. 5.1.2).

For all remaining levels, we observed that the memory capacity of the accelerator is ample when the HKS method is applied. This observation suggests that we can potentially reduce the overall computational overhead when employing FW-KDS by increasing its **evk** size. Fig. 10(a) compares the computational cost and **evk** size of the FW-KDS method with $\alpha = 6$, $\alpha' = 10$, against the HKS method with $\alpha = 12$. The FW-KDS method exhibits lower computational overhead than the HKS approach for levels ranging from 29 to 13, except for five levels between 24 and 20. For these levels, during the bootstrapping process, due to the utilization of double-prime scaling, key-switching computations are only present at levels 23 and 21. Fig. 10(b) also provides the working set size for the FW-KDS method, confirming that its storage demand remains within the memory capacity. Therefore, for levels 29 to 14, we employ the FW-KDS method to further reduce the computational burden. These levels correspond precisely to the EvalMod process, which involves a significant number of `HMult` operations. For levels below 13, increasing the **evk** size does not yield computational cost reductions through the FW-KDS method. Therefore, the HKS method is retained for these lower levels.

# 6 HAWK Architecture

## 6.1 Overview

HAWK consists of four clusters (Fig. 11(a)), each featuring a lane-wise network-on-chip (NoC) and adhering to a unified global data distribution policy. Each cluster integrates several key functional units, including the NTTU, which performs fully pipelined (I)NTT operations; the AutoU, responsible for automorphism operations required in rotation using efficient permutation logic; and the element-wise unit, which handles operations such as ciphertext multiplication and double-prime rescaling. To accommodate the exact base conversion introduced by our algorithmic optimizations, HAWK redesigns the base conversion unit to support both the enhanced and conventional BConv operations. Furthermore, an on-chip pseudo random number generator (PRNG) is incorporated to generate half of evaluation keys on the fly. This design choice reduces on-chip memory requirements and alleviates bandwidth limitations.

HAWK aims to support diverse key-switching methods, specifically, `bs`: HKS + `gs`: MFW-KDS for high-level H-IDFT, FW-KDS for medium-level EvalMod, and HKS for low-level computations (Sec. 5). Prior FHE-oriented accelerators, such as ARK and SHARP, have optimized extensively for the HKS-based implementations. We take SHARP as a starting point for our design to efficiently support HKS, and present necessary architectural modifications to accommodate the computational demands of FW-KDS at the hardware level.

Fig. 11(b) and (c) illustrate the execution flow and data movement among HAWK hardware components during the processes of `bs`: HKS and `gs`: MFW-KDS. During HKS execution, the rounding circuit within the EBConvU (Sec. 6.3) is explicitly bypassed to functionally complete `BConv` (Eq. 7). For MFW-KDS, the inner product computation is performed within the EWU (comprising the MADU and DSU, Sec. 6.4). Half of the polynomials processed by EWU are written directly back to local memory, bypassing the subsequent ring conversion process. This ensures that these polynomials remain in $\mathcal{R}_H$ throughout the $(\mathtt{gs} - 1)$ `HRot` operations (except for the last one) within the H-IDFT giant step (Sec. 5.1.1).

## 6.2 NTTU

This module executes the (I)NTT operation fully pipelined using a ten-step algorithm, featuring four butterfly execution groups, two quad transposition units, and a large transposition buffer for vectorized computations. Each butterfly group contains 256 lanes, grouped into 16 lane groups, functioning as Montgomery modular multipliers. The quad transposition units perform data transpositions within lane groups, while the transposition buffer handles global data transpositions. Polynomials are organized in a $16 \times 16 \times 16 \times 16$ format, aligned with the 256-element vector layout of SHARP, facilitating SIMD-style vector lane mapping. The (I)NTT operation is executed in two pipelined stages: the first processes data within

$16 \times 16$ sub-blocks, reducing data movement, and the second performs global butterfly operations after quad transpose. This design reduces on-chip memory bandwidth and communication overhead, resulting in an efficient and scalable (I)NTT execution pipeline.

## 6.3 Exact Base Conversion Unit (EBConvU)

As analyzed in Sec. 3.4, the exact base conversion ExactBConv (Eq. 12) in KDS (including the proposed FW-KDS) comprises a hardware-unfriendly rounding operation. To constrain the rounding error, its hardware implementation necessitates high-width multipliers and large-width accumulation units. Moreover, the intrinsic rounding error is tied to the operand bit width and cannot be completely avoided. To address these limitations, we propose a novel, hardware-friendly computational approach, requiring only small bit-width multiplication and addition operations. More importantly, this new approach eliminates the rounding computational error.

Lemma 6.1. *For positive real numbers* $\{\frac{a_i}{q_i}\}_{0\le i<k}$ *and an integer* $u$, *the rounding error satisfies* $|\sum_{i=0}^{k-1}\frac{a_i}{q_i}-u| < \frac{1}{4}$. *If* $2^r \ge \delta\cdot k\cdot max\{q_i\}$, $2^\tau \ge \frac{2\delta}{\delta-2}\cdot k$, $\delta > 2$, *then* $u = \left\lfloor \frac{3}{4} + 2^{-\tau}\cdot\sum_{i=0}^{k-1}\left\lfloor a_i\cdot\left\lfloor\frac{2^r}{q_i}\right\rfloor/2^{r-\tau}\right\rfloor\right\rfloor$.

Proof. Since $|\sum_{i=0}^{k-1}\frac{a_i}{q_i}-u| < \frac{1}{4}$, we have

$$u-\frac{1}{4} < \sum_{i=0}^{k-1}\frac{a_i}{q_i} < u+\frac{1}{4}.$$

Given $2^r \ge \delta\cdot k\cdot max\{q_i\}$ and $2^\tau \ge \frac{2\delta}{\delta-2}\cdot k$, then

$$\frac{k\cdot max\{a_i\}}{2^r} \le \frac{1}{\delta},\ \frac{k}{2^\tau} \le \frac{\delta-2}{2\delta}$$

Denote $g = \frac{3}{4} + 2^{-\tau}\cdot\sum_{i=0}^{k-1}\left\lfloor a_i\cdot\left\lfloor\frac{2^r}{q_i}\right\rfloor/2^{r-\tau}\right\rfloor$. We have

$$g \le \frac{3}{4} + 2^{-\tau}\cdot\sum_{i=0}^{k-1} a_i\cdot\frac{2^r}{q_i}/2^{r-\tau} = \frac{3}{4}+\sum_{i=0}^{k-1}\frac{a_i}{q_i} < \frac{3}{4}+u+\frac{1}{4}$$

$$\begin{aligned} g &\ge \frac{3}{4} + 2^{-\tau}\cdot\sum_{i=0}^{k-1}\left\{a_i\cdot\left(\frac{2^r}{q_i}-1\right)/2^{r-\tau}-1\right\} \\ &= \frac{3}{4}+\sum_{i=0}^{k-1}\frac{a_i}{q_i}-\sum_{i=0}^{k-1}\frac{a_i}{2^r}-\frac{k}{2^\tau} \\ &\ge \frac{3}{4}+u-\frac{1}{4}-\frac{k\cdot max\{a_i\}}{2^r}-\frac{k}{2^\tau} \ge u+\frac{1}{2}-\frac{1}{\delta}-\frac{\delta-2}{2\delta} = u \end{aligned}$$

Thus, $u \le g < u+1$. Finally, $u = \lfloor g\rfloor$. □

Therefore, when the sum $\sum_{i=0}^{k-1} x_i/q_i$ deviates from an integer $u$ by less than 1/4 (namely the rounding offset), we can leverage Lemma 6.1 to swiftly and accurately compute this integer based on the values of $x_i/q_i$. For computation, the expression $\lfloor\frac{2^r}{q_i}\rfloor$ is a pre-computed constant integer. By choosing a small value for $\delta$, we can ensure that $\lfloor\frac{2^r}{q_i}\rfloor$ remains a small-width number. The floor operation in the calculation formula for the integer $u$ corresponds to a bit truncation operation in the hardware circuit. As a result, the entire computation process $\left\lfloor\frac{3}{4}+2^{-\tau}\cdot\sum_{i=0}^{k-1}\left\lfloor a_i\cdot\left\lfloor\frac{2^r}{q_i}\right\rfloor/2^{r-\tau}\right\rfloor\right\rfloor$ involves only small-width multiplications, accumulations, and bit truncation operations. Thus, this computational approach is friendly and advantageous for hardware implementation.

When applying this lemma to the FW-KDS (KDS) method, we can set $H > 4\cdot 2B_d$ in modulus selection (Sec. 4.1). Then, the target rounded integer $z$ (see Eq. 13) satisfies:

$$\left\lfloor\sum_{i=0}^{\alpha'-1}[A]_{h_i}/h_i\right\rfloor = \left\lfloor\frac{[A]_H + z\cdot H}{H}\right\rfloor = \left\lfloor\frac{[A]_H}{H}+z\right\rfloor = z \qquad (15)$$

In this case, the offset satisfies $\frac{[A]_H}{H} < \frac{2B_d}{H} < \frac{1}{4}$. Then the rounding deviation satisfies $|\sum_{i=0}^{\alpha'-1}[A]_{h_i}/h_i - z| = |\frac{[A]_H}{H}| < \frac{1}{4}$. Thus we can utilize the Lemma 6.1 to accurately compute the rounded integer $z$.

In our implementation, $\delta$ is set to 4. Then, $2^r \ge 4k\cdot max\{h_i\}$, $2^\tau \ge 4k$. For the FW-KDS method, $k = \alpha'$ is at most 6 bits. Therefore, $\lfloor\frac{2^r}{q_i}\rfloor$ is at most 17 bits. Fig. 11(d) and (e) illustrate the circuit-level implementations of the traditional rounding method (Eq. 13) and our proposed exact approach, respectively. As shown in Fig. 11(d), to ensure that the rounding error remains below $2^{-30} = 2^{-72+36+6}$, the traditional method consumes multipliers and adders with large bit-widths, specifically 72bit×36bit, and 108bit+114bit, respectively. In contrast, as depicted in Fig. 11(e), at the circuit implementation level, our proposed approach not only eliminates the computational error (Lemma 6.1), but also only necessitates multipliers and adders with small bit widths, i.e., 17bit×36bit and 17bit+23bit, respectively. This effectively reduces the hardware cost of functional units. For the BConv, we adopt the similar two-dimensional systolic array architecture as SHARP to implement this functional unit.

## 6.4 Element-Wise Unit (EWU)

Beyond NTT, INTT, BConv, and automorphism, the remaining primary HE functions are element-wise computations. To flexibly support a variety of element-wise functions, we design a versatile functional component, MADU. Specifically, the MADU consists of four Barrett modular multipliers, two adders, a control unit, and an input-output buffer for synchronization. This work adopts a 36-bit machine word length. To preserve ciphertext precision, especially during bootstrapping, a double-prime rescaling mechanism is necessary. A double-prime scaling unit (DSU) is adopted for this purpose as in SHARP. This unit comprises two modular multipliers and a modular adder. During the inner product process of FW-KDS, each $[a_i]_{\mathcal{H}}$ undergoes $\gamma$ multiplications with **evk**. To facilitate this, the four modular multipliers included in MADU are configured with independent input and output capabilities, enabling simultaneous support for two pairs of multiplications with **evk**. In our implementation, $\gamma$ is at most 3. Therefore, during the computation of the inner product in FW-KDS, the DSU is also repurposed for supporting an additional pair of multiplications with **evk**. Similarly, the two modular multipliers within the DSU are configured as independent entities, while retaining the accumulation capability for the double-prime scaling operation. To optimize inner product support, we strategically consolidate the MADU and DSU into a cohesive functional unit that enables seamless data distribution and efficient operational scheduling. This unified computational component is designated as the Element-Wise Unit (EWU).

## 6.5 Memory Organization

HAWK employs a heavily banked memory architecture, organized into multiple banks per lane group and further subdivided into

**Table 2: Hardware configuration of FHE accelerators. Cap. denotes capacity (MB), BW denotes bandwidth (TB/s).**

| | CraterLake | ARK | Taiyi | SHARP | HAWK |
|---|---|---|---|---|---|
| Word length | 28 bit | 64 bit | 36 bit | 36 bit | 36 bit |
| Lanes | 2048 | 1024 | 1024 | 1024 | 1024 |
| Technology | 12nm | 7nm | 7nm | 7nm | 7nm |
| Frequency | 1GHz | 1GHz | 1GHz | 1GHz | 1GHz |
| Off-chip BW | 1 | 1 | 1 | 1 | 1 |
| On-chip BW | 256+26 | 20+72 | 75 | 36+36 | 36+45 |
| Memory Cap. | 84 | 512+76 | 128.25 | 180+18 | 192+20 |
| Area ($mm^2$) | 472.3 | 418.3 | 151.6 | 178.8 | 186.4 |

**Table 3: Comparison of performance between HAWK and other prior FHE accelerators (ms).**

| Workloads | CraterLake | BTS | ARK | Taiyi§ | SHARP | HAWK |
|---|---|---|---|---|---|---|
| Bootstrapping | 6.33 | 22.88 | 3.52 | 7.61 | 3.12 | 2.15 |
| HELR256 | 3.81 | - | - | - | 1.82 | 1.36 |
| HELR1024 | - | 28.4 | 7.42 | 4.14 | 2.53 | 1.96 |
| Sorting | - | 15.6 s† | 1.99 s‡ | - | 1.38 s | 0.98 s |
| ResNet-20 | 321 | 1910 | 125 | 162.6 | 99 | 74.89 |

† Sorting performance of BTS is sourced from SHARP [34].
‡ Performance of sorting on 16,384 elements is also sourced from SHARP [34].
§ Performance is sourced from $Taiyi_{Serial}$. Area for $Taiyi_{Parallel}$ is not reported as its memory capacity (128.25 MB) is insufficient to support parallel computing.

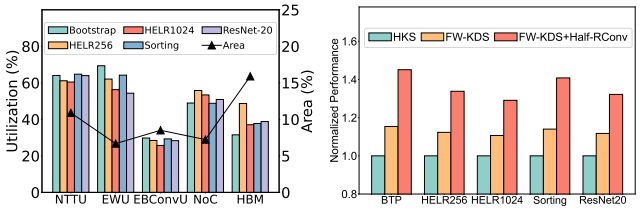
sub-banks. Through memory interleaving, this design enables concurrent handling of four reads and four writes per lane per cycle. Sequential data access, exploiting the locality of single-limb operations, eliminates bank conflicts and further simplifies control and addressing. To accommodate the memory demands of the HKS+MFW-KDS for H-IDFT and the FW-KDS for EvalMod, HAWK incorporates a scratchpad memory capacity of 192 MB (Fig. 10(b)).

# 7 Evaluation

## 7.1 Experimental Setup

We implemented HAWK's components in RTL and synthesized them leveraging the ASAP7 library [17, 18], a predictive process design kit specifically developed for the 7nm technology node. The evaluation of the scratchpad (192MB) and register files (20MB) was carried out using FinCACTI [47]. Two HBM stacks are utilized in HAWK, providing a total of 1TB/s off-chip memory bandwidth. Its area and power consumption were estimated based on [31, 42]. The functional units are fully pipelined and operate at a frequency of 1 GHz. The SRAM components are designed as single-ported modules and run double-pumped at 2GHz. This configuration enables concurrent execution of read and write operations per cycle. Table 2 summarizes the specific hardware configuration for HAWK.

We built a cycle-accurate simulator to evaluate HAWK performance. Based on RTL implementations, we first model the capability and latency of each component, along with varying bandwidth. The simulator takes as input an FHE program written in Python-embedded DSL and translates it into a dataflow graph, breaking down complex functions into primary operations. Based on the derived computation and data dependencies, the simulator assigns operations to hardware components, and schedules their execution and data movements with cycle-level timing precision.




**(a) Component utilization and area (b) Performance on workloads.**

**Figure 12: (a) Utilization of components, and their respective portions in the accelerator area. (b) Performance changes from the gradual addition of FW-KDS and Half-RConv.**

We used the following representative workloads for evaluation.

- **Bootstrapping:** We conducted the performance evaluation of full-slot bootstrapping under the parameters: $L = 35$, $\mathsf{dnum} = 3$, $L_{\text{eff}} = 8$, and $\log QP \leq 1555$.
- **HELR:** HELR[28] represents the CKKS implementation of logistic regression training for binary classification. The model was trained over 32 iterations, and we reported the average latency per iteration. For complete analysis, we evaluated batch sizes of 256 (HELR256) and 1,024 (HELR1024).
- **Sorting:** This is a two-way bitonic sorting based on the CKKS scheme, which orders an array with 16,384 elements [30].
- **ResNet-20:** It implements the homomorphic ResNet-20 developed by Lee et al. [38], performing a convolutional neural network inference with a $32 \times 32 \times 3$ CIFAR-10 image.

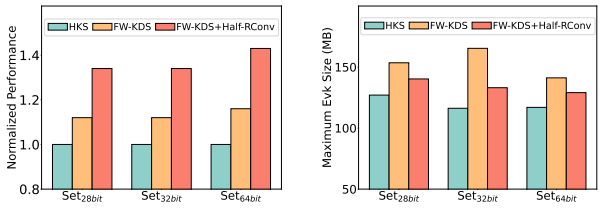
As analyzed in Sec. 5, we apply FW-KDS to the high (H-IDFT, bs: HKS + gs: MFW-KDS) and medium (EvalMod: FW-KDS) levels. The remaining low levels for H-DFT and applications select HKS.

## 7.2 Performance Comparison

Table 3 provides the performance comparisons. HAWK achieves a 16.64×, 3.34×, 2.28×, and 1.36× performance improvement in gmean, compared to BTS, CraterLake, ARK, and SHARP, respectively. A significant portion of the execution time for these workloads is dominated by bootstrapping, accounting for up to 94.8% for the sorting task, and averaging 82.1% across varying workloads.

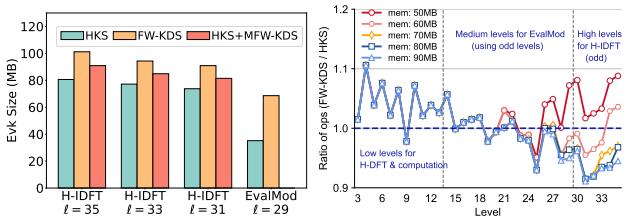
This substantial overhead highlights the critical importance of efficient bootstrapping acceleration in the field of FHE. HAWK enhances its performance primarily by integrating the proposed FW-KDS method for key-switching at the high and medium levels, particularly during the H-IDFT and EvalMod processes. Additionally, the memory requirements for the key-switching key can be significantly reduced by utilizing unified shorter word lengths for the evaluation key. Furthermore, HAWK adopts the proposed Half-RConv technique for H-IDFT computations, further optimizing both computational and memory demands. Through these optimizations, HAWK deploys the enhanced key decomposition switching on the FHE specialized accelerator with only a modest 14MB increase in on-chip memory capacity compared to SHARP. With a negligible 4% area expansion, HAWK achieves performance improvements of 1.45×, 1.34×, 1.29×, 1.41×, and 1.32× in bootstrapping, HELR256, HELR1024, sorting, and ResNet-20 tasks, respectively.

The related work Taiyi [20] directly replaces the HKS method with the KDS approach across all levels at a short word length




(a) Performance of bootstrapping (b) Evaluation key size

**Figure 13: (a) Performance changes from the gradual addition of FW-KDS and Half-RConv on different parameter sets. (b) Corresponding evaluation key size at the maximum level.**




(a) Key size evaluation (b) FW-KDS vs HKS

**Figure 14: (a) Key size evaluation for H-IDFT and EvalMod. H-IDFT is divided into three sub-routines at levels 35, 33, 31 due to double-prime scaling. EvalMod features evk reusing. Size of the maximum evk for EvalMod at level $\ell = 29$ is shown (lower levels reuse this evk). (b) Complexity comparisons of FW-KDS and HKS under varying available memory (mem) for evk. The ratio of operations is computed as FW-KDS/HKS.**

of 36 bits. This straightforward employment of KDS method for hardware implementation leads to dramatic increases both in computational complexity and **evk** memory requirements, as analyzed in Sec. 3.1 and Sec. 3.3. Consequently, HAWK demonstrates superior performance, yielding 3.54×, 2.11×, and 2.17× speedups over Taiyi for bootstrapping, HELR1024, and ResNet-20, respectively.

## 7.3 Utilization of Hardware Components

Fig. 12(a) presents the utilization of primary functional components in HAWK across diverse workloads, indicating that the optimized key-switching method remains predominantly compute-bound. The NTTU handles the conversion of ciphertexts between their coefficient and evaluation representations. Although the proposed FW-KDS and MFW-KDS methods reduce the overall computational overhead of expensive (I)NTT operations, these transformations still account for a significant portion of the overall workload. As a result, the NTTU achieves high utilization across different HE workloads, with an average active utilization rate of 62.86%. In the FW-KDS method, the ring conversions are introduced, and the cost of inner product with the expanded evaluation key is increased. Consequently, the EBConvU and EWU components in HAWK exhibit average utilization rates of 28.3% and 61.24%, respectively.

## 7.4 Ablation Study

To quantify the contribution of our key-switching optimizations, we conduct an ablative analysis across various HE workloads. We establish a baseline by using each task that exclusively relies on the HKS method for key-switching, and then progressively integrate our proposed optimizations. Fig. 12(b) highlights the performance gains achieved by each optimization technique across different HE applications. Introducing our FW-KDS method in place of the HKS approach at the middle-to-high levels yields a 1.11× to 1.15× performance speedup across applications. Furthermore, when Half-RConv technique is applied to FW-KDS, the performance of these applications further increases by 1.19× to 1.26×. This enhancement is primarily attributed to the application of Half-RConv at the highest levels, which effectively reduces the ring conversion computations by half during the H-IDFT process—a stage that involves numerous expensive INTT, NTT, and exact base conversion operations, thereby significantly boosting overall performance.

## 7.5 Sensitivity Study to FHE Parameter Sets

Previous HE accelerators have employed varying machine word lengths. For instance, CraterLake [46] utilizes a 28-bit word length, while ARK [35] opts for 64 bits. The GPU acceleration Cheddar [33] adopts the 32-bit word length, conforming to the GPU hardware characteristics. SHARP [34] analyzed the impact of word length selection and discovered that 36 bits are sufficient to support representative HE workloads robustly and efficiently. Therefore, HAWK also adopts a machine word length of 36 bits.

We evaluate the acceleration effects of the proposed FW-KDS and Half-RConv optimizations under HE parameter sets with different word lengths. Similarly, we use bootstrapping that employs only the HKS method for key-switching as the baseline, and then progressively incorporate the proposed optimizations. When evaluating across parameter sets with different word lengths, we base our experiments on HAWK, requiring only adjustments to the machine word length of the functional units and modifications to the on-chip memory capacity to accommodate the maximum evaluation keys required for the key-switchings at the highest level $L$. Parameter sets with word lengths of 28, 32, and 64 bits are referred to as $\texttt{Set}_{28/32/64bit}$. The corresponding values of $L$ are 57, 54, and 23, respectively, while parameter `dnum` remains constant at 4.

Fig. 13 illustrates the performance improvements for different word-length parameter sets, along with the maximum size of the evaluation keys required for their baby and giant steps. By applying FW-KDS, performance improves by 1.12×/1.12×/1.16× on $\texttt{Set}_{28/32/64bit}$ compared to using HKS. Further incorporating Half-RConv yields additional performance gains of 1.22×/1.22×/1.25×. The acceleration effects of FW-KDS and Half-RConv become more pronounced under parameter sets with larger word lengths. When combining FW-KDS and Half-RConv, the size of the corresponding evaluation key at the highest level increases modestly by only 13.2/16.9/12.1MB for $\texttt{Set}_{28/32/64bit}$, while achieving overall performance improvements of 1.34×/1.34×/1.43× over HKS alone.

## 7.6 Key Size Evaluation

As analyzed in Sec. 5, MFW-KDS combined with HKS (FW-KDS) is employed at high (medium) levels for the H-IDFT (EvalMod) process. The remaining low levels for H-DFT and applications still utilize HKS (Sec. 5.2). Hence, we focus on the evaluation key sizes for H-IDFT and EvalMod across different key-switching methods. Fig. 14(a) shows these key sizes. For H-IDFT at levels $\ell = 35/33/31$, HKS

combined with MFW-KDS yields **evk** sizes of 90.89/84.89/81.46MB, with an increase of 10.29/7.72/7.72MB versus HKS, respectively. For EvalMod that features key reusing across levels, its maximum **evk** size ($\ell = 29$) reaches 68.6MB (lower levels reuse this **evk**), a 33.44 MB increase relative to HKS. This increase in key size still resides in the memory capacity (Sec. 5.2). The on-chip memory capacity of HE accelerator is primarily dependent on the highest-level key size. In HAWK, only 10.29MB is increased versus HKS at the highest level (see Fig. 9(b)). For other FHE parameter sets, their highest-level key sizes are presented in Sec. 7.5 (see Fig. 13(b)).

### 7.7 Impact of $H > 4 \cdot 2B_d$ on Complexity

In the FW-KDS method, the converted modulus $H > 2B_d$ is typically constructed from the moduli of $max\{\tilde{Q}_k\}$ and $max\{Q_j\}$, and an additional modulus $q_{i'} > \mathsf{dnum} \cdot N/2$ (Sec. 4.1). The proposed computation approach for exact base conversion (Sec. 6.3) requires a stronger condition of $H > 4 \cdot 2B_d$. This threshold is exactly $4 \cdot 2B_d = \mathsf{dnum} \cdot 2N \cdot max\{\tilde{Q}_k\} \cdot max\{Q_j\}$ (Eq. 10). As $\mathsf{dnum} < L << 128$ and $N = 2^{16}$, the bit width of $\mathsf{dnum} \cdot 2N$ is at most 24 bits. Moreover, the bit width of moduli $q_{i'}$ for CKKS is at least 28 bits [46] (HAWK uses 36 bits), then we have $q_{i'} > \mathsf{dnum} \cdot 2N$. Consequently, the converted modulus $H$ already surpasses $4 \cdot 2B_d$. Therefore, $H$ remains unchanged and does not incur any increase in computational complexity.

## 8 Discussion

### 8.1 Selection of Key-Switching Methods

The computational overhead of FW-KDS correlates with available memory capacity for storing **evk**. Fig. 14(b) presents comparisons of FW-KDS and HKS under varying **evk** memory capacities ($L = 35$, $\alpha = 12$). The memory demand for **evk** at level 35 in HKS reaches 40.3MB. At high levels, FW-KDS provides computational benefits over HKS with increased memory demands. For medium odd levels (double-prime scaling), FW-KDS outperforms HKS in most cases as key-switching becomes less memory-heavy. Conversely, at low levels, HKS proves more efficient as the introduced ring conversions in FW-KDS dominate. In summary: **(i)** For H-IDFT at high levels, we prefer FW-KDS for key-switching, albeit at an increased memory demand for **evk**. Importantly, MFW-KDS (Sec. 5.1) is strongly recommended to halve the involved expensive ring conversions. **(ii)** For EvalMod featuring **evk** reusing at medium levels, FW-KDS is also preferred as key-switchings in these levels are more compute-heavy. **(iii)** HKS is preferred for computations at lower levels. These method combinations deliver a 1.34× to 1.45× speedup in bootstrapping performance with only a modest increase in memory demands for holding evaluation keys (Sec. 7.4 and 7.5).

### 8.2 Adaptation to Other FHE Schemes

This work focuses on the RLWE-based CKKS scheme. Other RLWE-based FHE schemes, typically such as BGV [8, 27] and BFV [6, 19], employ similar key-switching processes. Thus the proposed FW-KDS method and relevant optimizations can be potentially adapted to accelerate BGV and BFV implementations. While for LWE-based schemes like TFHE [15, 16] towards homomorphic boolean arithmetic, our method is inapplicable as TFHE features a fundamentally different key-switching mechanism.

### 8.3 Combining FW-KDS with Hoisting

The hoisting technique [7, 27] can effectively reduce the H-(I)DFT computational complexity at the expense of a multiplicative increase in **evk** size. Additionally, this technique is independent of the specific key-switching method employed. For hardware implementations, hoisting incurs a significant increase in either on-chip memory demands or off-chip data access. To alleviate memory bottlenecks, ARK [35] proposes the minimum key-switching strategy by reusing a single **evk** throughout the baby (giant) step instead of memory-intensive hoisting. For the same reason, SHARP [34] and our work follow the minimum key-switching strategy. While for GPU implementations [32, 33], the hoisting technique is commonly leveraged with **evk** stored in large global memory. Combining the proposed FW-KDS method with hoisting offers a potential avenue for optimizing GPU-based FHE performance.

## 9 Related Work

Prior efforts have alleviated FHE performance bottlenecks in several ways. FAB [2] introduces datapath modifications for key-switching to reduce the number of NTTs and memory traffic. MAD [1] proposes specialized caching strategies for different kernels within the key-switching process to mitigate memory bottlenecks. ARK [35] addresses the substantial on-chip memory consumption for H-(I)DFT by introducing the minimum key-switching strategy, thereby significantly decreasing the required evaluation keys. SHARP [34] exploits the area-efficient short word-length components to achieve high computational efficiency without compromising accuracy in representative FHE applications. FPGA-based accelerations have also been explored [2, 52], but their performance remains constrained by the inherent limitations of FPGA available resources. In addition, several efforts have sought to leverage the massive parallelism of GPUs to accelerate CKKS computations [3, 21, 32, 33, 51].

## 10 Conclusion

This work presents HAWK, an accelerator co-designed with algorithmic optimizations for efficient FHE. Our analysis reveals that the original KDS method imposes substantial computational overhead and on-chip memory requirements, rendering it unsuitable for hardware implementation. To address this, we introduce (M)FW-KDS, an architecture-friendly key-switching method that avoids dynamic word-length changes, aligning better with fixed-word-length hardware. We further employ a dual-mode strategy combining FW-KDS with traditional HKS to balance computational performance and memory footprint. To meet the precision requirements of FW-KDS, we develop a novel hardware-friendly computational approach for exact base conversion. Through this hardware-algorithm co-design, HAWK achieves up to a 1.45× performance improvement.

## Acknowledgments

We thank the anonymous reviewers for their constructive comments. We also thank Yisong Chang for valuable discussions and suggestions that improved the manuscript, and Zhen Gu for the insight that inspired our general approach to the rounding operation.

## References


[1] Rashmi Agrawal, Leo De Castro, Chiraag Juvekar, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2023. MAD: Memory-Aware Design Techniques for Accelerating Fully Homomorphic Encryption. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 685–697. https://doi.org/10.1145/3613424.3614302

[2] Rashmi Agrawal, Leo de Castro, Guowei Yang, Chiraag Juvekar, Rabia Tugce Yazicigil, Anantha P. Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. 2023. FAB: An FPGA-based Accelerator for Bootstrappable Fully Homomorphic Encryption. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*. IEEE, 882–895. https://doi.org/10.1109/HPCA56546.2023.10070953

[3] Ahmad Al Badawi, Yuriy Polyakov, Khin Mi Mi Aung, Bharadwaj Veeravalli, and Kurt Rohloff. 2021. Implementation and Performance Evaluation of RNS Variants of the BFV Homomorphic Encryption Scheme. *IEEE Transactions on Emerging Topics in Computing* 9, 2 (2021), 941–956. https://doi.org/10.1109/TETC.2019.2902799

[4] Ahmad Al Badawi, Bharadwaj Veeravalli, Chan Fook Mun, and Khin Mi Mi Aung. 2018. High-Performance FV Somewhat Homomorphic Encryption on GPUs: An Implementation using CUDA. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2018, 2 (May 2018), 70–95. https://doi.org/10.13154/tches.v2018.i2.70-95

[5] Jean-Claude Bajard, Julien Eynard, M. Anwar Hasan, and Vincent Zucca. 2016. A Full RNS Variant of FV Like Somewhat Homomorphic Encryption Schemes. In *Selected Areas in Cryptography - SAC 2016 - 23rd International Conference, St. John's, NL, Canada, August 10-12, 2016, Revised Selected Papers*, Vol. 10532. Springer, 423–442. https://doi.org/10.1007/978-3-319-69453-5_23

[6] Jean-Claude Bajard, Julien Eynard, Anwar Hasan, and Vincent Zucca. 2016. A Full RNS Variant of FV like Somewhat Homomorphic Encryption Schemes. Cryptology ePrint Archive, Paper 2016/510. https://eprint.iacr.org/2016/510

[7] Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2021. Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-sparse Keys. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Anne Canteaut and François-Xavier Standaert (Eds.). Springer, Cham, 587–617.

[8] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) Fully Homomorphic Encryption without Bootstrapping. *ACM Transactions on Computing Theory* 6, 3, Article 13 (July 2014), 36 pages. https://doi.org/10.1145/2633600

[9] Zvika Brakerski and Vinod Vaikuntanathan. 2011. Fully Homomorphic Encryption from Ring-LWE and Security for Key Dependent Messages. In *Advances in Cryptology - CRYPTO 2011 - 31st Annual Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2011. Proceedings*, Vol. 6841. Springer, 505–524. https://doi.org/10.1007/978-3-642-22792-9_29

[10] Hao Chen, Ilaria Chillotti, and Yongsoo Song. 2019. Improved Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2019: 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19–23, 2019, Proceedings, Part II* (Darmstadt, Germany). Springer-Verlag, Berlin, Heidelberg, 34–54. https://doi.org/10.1007/978-3-030-17656-3_2

[11] Hao Chen, Kim Laine, and Rachel Player. 2017. Simple Encrypted Arithmetic Library - SEAL v2.1. In *Financial Cryptography and Data Security - FC 2017 International Workshops, WAHC, BITCOIN, VOTING, WTSC, and TA, Sliema, Malta, April 7, 2017, Revised Selected Papers*, Vol. 10323. Springer, 3–18. https://doi.org/10.1007/978-3-319-70278-0_1

[12] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2018*, Jesper Buus Nielsen and Vincent Rijmen (Eds.). Springer International Publishing, Cham, 360–384.

[13] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. A Full RNS Variant of Approximate Homomorphic Encryption. In *Selected Areas in Cryptography - SAC 2018 - 25th International Conference, Calgary, AB, Canada, August 15-17, 2018, Revised Selected Papers*, Vol. 11349. Springer, 347–368. https://doi.org/10.1007/978-3-030-10970-7_16

[14] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. 2017. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, Vol. 10624. Springer, 409–437. https://doi.org/10.1007/978-3-319-70694-8_15

[15] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2016. Faster Fully Homomorphic Encryption: Bootstrapping in Less Than 0.1 Seconds. In *Advances in Cryptology – ASIACRYPT 2016*, Jung Hee Cheon and Tsuyoshi Takagi (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 3–33.

[16] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *Journal of Cryptology* 33, 1 (Jan. 2020), 34–91. https://doi.org/10.1007/s00145-019-09319-x

[17] Lawrence T. Clark, Vinay Vashishtha, David M. Harris, Samuel Dietrich, and Zunyan Wang. 2017. Design flows and collateral for the ASAP7 7nm FinFET predictive process design kit. In *2017 IEEE International Conference on Microelectronic Systems Education, MSE 2017, Lake Louise, AB, Canada, May 11-12, 2017*. IEEE, 1–4. https://doi.org/10.1109/MSE.2017.7945071

[18] Lawrence T. Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandarasekaran Ramamurthy, and Greg Yeric. 2016. ASAP7: A 7-nm finFET predictive process design kit. *Microelectron. J.* 53 (2016), 105–115. https://doi.org/10.1016/J.MEJO.2016.04.006

[19] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat Practical Fully Homomorphic Encryption. Cryptology ePrint Archive, Paper 2012/144. https://eprint.iacr.org/2012/144

[20] Shengyu Fan, Xianglong Deng, Zhuoyu Tian, Zhicheng Hu, Liang Chang, Rui Hou, Dan Meng, and Mingzhe Zhang. 2024. Taiyi: A high-performance CKKS accelerator for Practical Fully Homomorphic Encryption. arXiv:2403.10188 [cs.CR] https://arxiv.org/abs/2403.10188

[21] Shengyu Fan, Zhiwei Wang, Weizhi Xu, Rui Hou, Dan Meng, and Mingzhe Zhang. 2023. TensorFHE: Achieving Practical Computation on Encrypted Data Using GPGPU. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*. IEEE, 922–934. https://doi.org/10.1109/HPCA56546.2023.10071017

[22] Robin Geelen, Michiel Van Beirendonck, Hilder Pereira, Brian Huffman, Tynan McAuley, Ben Selfridge, Daniel Wagner, Georgios Dimou, Ingrid Verbauwhede, Frederik Vercauteren, and David Archer. 2023. BASALISC: Programmable Hardware Accelerator for BGV Fully Homomorphic Encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (08 2023), 32–57. https://doi.org/10.46586/tches.v2023.i4.32-57

[23] Craig Gentry, Shai Halevi, and Nigel P. Smart. 2012. Homomorphic Evaluation of the AES Circuit. In *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, Vol. 7417. Springer, 850–867. https://doi.org/10.1007/978-3-642-32009-5_49

[24] Shai Halevi, Yuriy Polyakov, and Victor Shoup. 2019. An Improved RNS Variant of the BFV Homomorphic Encryption Scheme. In *Topics in Cryptology - CT-RSA 2019 - The Cryptographers' Track at the RSA Conference 2019, San Francisco, CA, USA, March 4-8, 2019, Proceedings*, Vol. 11405. Springer, 83–105. https://doi.org/10.1007/978-3-030-12612-4_5

[25] Shai Halevi and Victor Shoup. 2018. Faster Homomorphic Linear Transformations in HElib. In *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part I*, Vol. 10991. Springer, 93–120. https://doi.org/10.1007/978-3-319-96884-1_4

[26] Shai Halevi and Victor Shoup. 2020. Design and implementation of HElib: a homomorphic encryption library. *IACR Cryptol. ePrint Arch.* (2020), 1481. https://eprint.iacr.org/2020/1481

[27] Shai Halevi and Victor Shoup. 2021. Bootstrapping for HElib. *Journal of Cryptology* 34, 1 (Jan. 2021), 44 pages. https://doi.org/10.1007/s00145-020-09368-7

[28] Kyoohyung Han, Seungwan Hong, Jung Hee Cheon, and Daejun Park. 2019. Logistic Regression on Homomorphic Encrypted Data at Scale. In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*. AAAI Press, 9466–9471. https://doi.org/10.1609/AAAI.V33I01.33019466

[29] Kyoohyung Han and Dohyeong Ki. 2020. Better Bootstrapping for Approximate Homomorphic Encryption. In *Topics in Cryptology - CT-RSA 2020 - The Cryptographers' Track at the RSA Conference 2020, San Francisco, CA, USA, February 24-28, 2020, Proceedings*, Vol. 12006. Springer, 364–390. https://doi.org/10.1007/978-3-030-40186-3_16

[30] Seungwan Hong, Seunghong Kim, Jiheon Choi, Younho Lee, and Jung Hee Cheon. 2021. Efficient Sorting of Homomorphic Encrypted Data With k-Way Sorting Network. *IEEE Trans. Inf. Forensics Secur.* 16 (2021), 4389–4404. https://doi.org/10.1109/TIFS.2021.3106167

[31] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter C. Ma, Xiaoyu Ma, Thomas Norrie, Nishant Patil, Sushma Prasad, Cliff Young, Zongwei Zhou, and David A. Patterson. 2021. Ten Lessons From Three Generations Shaped Google's TPUv4i : Industrial Product. In *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Virtual Event / Valencia, Spain, June 14-18, 2021*. IEEE, 1–14. https://doi.org/10.1109/ISCA52012.2021.00010

[32] Wonkyung Jung, Sangpyo Kim, Jung Ho Ahn, Jung Hee Cheon, and Younho Lee. 2021. Over 100x Faster Bootstrapping in Fully Homomorphic Encryption through Memory-centric Optimization with GPUs. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 4 (2021), 114–148. https://doi.org/10.46586/TCHES.V2021.I4.114-148

[33] Jongmin Kim, Wonseok Choi, and Jung Ho Ahn. 2024. Cheddar: A swift fully homomorphic encryption library for cuda gpus. *arXiv preprint arXiv:2407.13055* (2024).

[34] Jongmin Kim, Sangpyo Kim, Jaewan Choi, Jaiyoung Park, Donghwan Kim, and Jung Ho Ahn. 2023. SHARP: A Short-Word Hierarchical Accelerator for Robust and Practical Fully Homomorphic Encryption. In *Proceedings of the 50th Annual*

*International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17-21, 2023*. ACM, 18:1–18:15. https://doi.org/10.1145/3579371.3589053

[35] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, Minsoo Rhu, John Kim, and Jung Ho Ahn. 2022. ARK: Fully Homomorphic Encryption Accelerator with Runtime Data Generation and Inter-Operation Key Reuse. In *55th IEEE/ACM International Symposium on Microarchitecture, MICRO 2022, Chicago, IL, USA, October 1-5, 2022*. IEEE, 1237–1254. https://doi.org/10.1109/MICRO56248.2022.00086

[36] Miran Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. 2023. Accelerating HE Operations from Key Decomposition Technique. In *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part IV*, Vol. 14084. Springer, 70–92. https://doi.org/10.1007/978-3-031-38551-3_3

[37] Sangpyo Kim, Jongmin Kim, Michael Jaemin Kim, Wonkyung Jung, John Kim, Minsoo Rhu, and Jung Ho Ahn. 2022. BTS: an accelerator for bootstrappable fully homomorphic encryption. In *ISCA '22: The 49th Annual International Symposium on Computer Architecture, New York, New York, USA, June 18 - 22, 2022*. ACM, 711–725. https://doi.org/10.1145/3470496.3527415

[38] Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. 2022. Low-Complexity Deep Convolutional Neural Networks on Fully Homomorphic Encryption Using Multiplexed Parallel Convolutions. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (Eds.). PMLR, 12403–12422. https://proceedings.mlr.press/v162/lee22e.html

[39] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2010. On Ideal Lattices and Learning with Errors over Rings. In *Advances in Cryptology - EUROCRYPT 2010, 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Monaco / French Riviera, May 30 - June 3, 2010. Proceedings*, Vol. 6110. Springer, 1–23. https://doi.org/10.1007/978-3-642-13190-5_1

[40] Amirhossein Ebrahimi Moghaddam, Buvana Ganesh, and Paolo Palmieri. 2024. Privacy-Preserving Sentiment Analysis Using Homomorphic Encryption and Attention Mechanisms. In *Applied Cryptography and Network Security Workshops: ACNS 2024 Satellite Workshops, AIBlock, AIHWS, AIoTS, SCI, AAC, SiMLA, LLE, and CIMSS, Abu Dhabi, United Arab Emirates, March 5–8, 2024, Proceedings, Part II* (Abu Dhabi, United Arab Emirates). Springer-Verlag, Berlin, Heidelberg, 84–100. https://doi.org/10.1007/978-3-031-61489-7_6

[41] Christian Vincent Mouchet, Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2020. Lattigo: A multiparty homomorphic encryption library in go. In *Proceedings of the 8th Workshop on Encrypted Computing and Applied Homomorphic Cryptography*. 64–70.

[42] Mike O'Connor, Niladrish Chatterjee, Donghyuk Lee, John M. Wilson, Aditya Agrawal, Stephen W. Keckler, and William J. Dally. 2017. Fine-grained DRAM: energy-efficient DRAM for extreme bandwidth systems. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2017, Cambridge, MA, USA, October 14-18, 2017*, Hillery C. Hunter, Jaime Moreno, Joel S. Emer, and Daniel Sánchez (Eds.). ACM, 41–54. https://doi.org/10.1145/3123939.3124545

[43] Jestine Paul, Meenatchi Sundaram Muthu Selva Annamalai, William Ming, Ahmad Al Badawi, Bharadwaj Veeravalli, and Khin Mi Mi Aung. 2021. Privacy-Preserving Collective Learning With Homomorphic Encryption. *IEEE Access* 9 (2021), 132084–132096. https://doi.org/10.1109/ACCESS.2021.3114581

[44] M. Sadegh Riazi, Kim Laine, Blake Pelton, and Wei Dai. 2020. HEAX: An Architecture for Computing on Encrypted Data. In *ASPLOS '20: Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, March 16-20, 2020*. ACM, 1295–1309. https://doi.org/10.1145/3373376.3378523

[45] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald G. Dreslinski, Christopher Peikert, and Daniel Sánchez. 2021. F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption. In *MICRO '21: 54th Annual IEEE/ACM International Symposium on Microarchitecture, Virtual Event, Greece, October 18-22, 2021*. ACM, 238–252. https://doi.org/10.1145/3466752.3480070

[46] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Nathan Manohar, Nicholas Genise, Srinivas Devadas, Karim Eldefrawy, Chris Peikert, and Daniel Sánchez. 2022. CraterLake: a hardware accelerator for efficient unbounded computation on encrypted data. In *ISCA '22: The 49th Annual International Symposium on Computer Architecture, New York, New York, USA, June 18 - 22, 2022*. ACM, 173–187. https://doi.org/10.1145/3470496.3527393

[47] Alireza Shafaei, Yanzhi Wang, Xue Lin, and Massoud Pedram. 2014. FinCACTI: Architectural Analysis and Modeling of Caches with Deeply-Scaled FinFET Devices. In *IEEE Computer Society Annual Symposium on VLSI, ISVLSI 2014, Tampa, FL, USA, July 9-11, 2014*. IEEE Computer Society, 290–295. https://doi.org/10.1109/ISVLSI.2014.94

[48] Kaustubh Shivdikar, Yuhui Bao, Rashmi Agrawal, Michael Shen, Gilbert Jonatan, Evelio Mora, Alexander Ingare, Neal Livesay, JosÉ L. AbellÁN, John Kim, Ajay Joshi, and David Kaeli. 2023. GME: GPU-based Microarchitectural Extensions to Accelerate Homomorphic Encryption. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 670–684. https://doi.org/10.1145/3613424.3614279

[49] Sujoy Sinha Roy, Furkan Turan, Kimmo Jarvinen, Frederik Vercauteren, and Ingrid Verbauwhede. 2019. FPGA-Based High-Performance Parallel Architecture for Homomorphic Computing on Encrypted Data. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 387–398. https://doi.org/10.1109/HPCA.2019.00052

[50] Furkan Turan, Sujoy Sinha Roy, and Ingrid Verbauwhede. 2020. HEAWS: An Accelerator for Homomorphic Encryption on the Amazon AWS FPGA. *IEEE Trans. Comput.* 69, 8 (2020), 1185–1196. https://doi.org/10.1109/TC.2020.2988765

[51] Hao Yang, Shiyu Shen, Wangchen Dai, Lu Zhou, Zhe Liu, and Yunlei Zhao. 2024. Phantom: A CUDA-Accelerated Word-Wise Homomorphic Encryption Library. *IEEE Transactions on Dependable and Secure Computing* 21, 5 (2024), 4895–4906. https://doi.org/10.1109/TDSC.2024.3363900

[52] Yinghao Yang, Huaizhi Zhang, Shengyu Fan, Hang Lu, Mingzhe Zhang, and Xiaowei Li. 2023. Poseidon: Practical Homomorphic Encryption Accelerator. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*. IEEE, 870–881. https://doi.org/10.1109/HPCA56546.2023.10070984

[53] Peng Zhang, Teng Huang, Xiaoqiang Sun, Wei Zhao, Hongwei Liu, Shangqi Lai, and Joseph K. Liu. 2023. Privacy-Preserving and Outsourced Multi-Party K-Means Clustering Based on Multi-Key Fully Homomorphic Encryption. *IEEE Transactions on Dependable and Secure Computing* 20, 3 (2023), 2348–2359. https://doi.org/10.1109/TDSC.2022.3181667

[54] Yilan Zhu, Xinyao Wang, Lei Ju, and Shanqing Guo. 2023. FxHENN: FPGA-based acceleration framework for homomorphic encrypted CNN inference. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 896–907. https://doi.org/10.1109/HPCA56546.2023.10071133